

Redes Neuronales Artificiales - Parte Dos

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



24 de febrero de 2026

Limitaciones del Perceptrón.

- ▶ El perceptrón resuelve problemas **linealmente separables**.
- ▶ Su frontera de decisión es siempre un hiperplano:

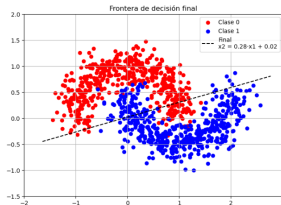
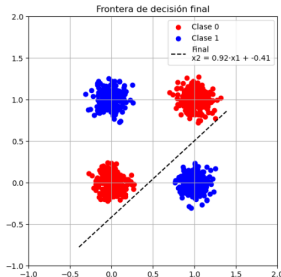
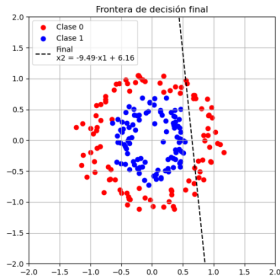
$$\hat{y} = H(\mathbf{w}^T \mathbf{x} + b)$$

donde H es la función escalón o identidad.

- ▶ No puede aprender funciones como XOR, que requieren una frontera de decisión no lineal.
- ▶ Necesitamos una arquitectura más flexible: **Redes Neuronales Multicapa**.

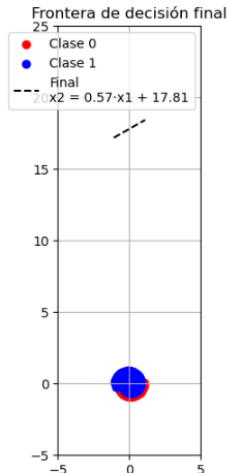
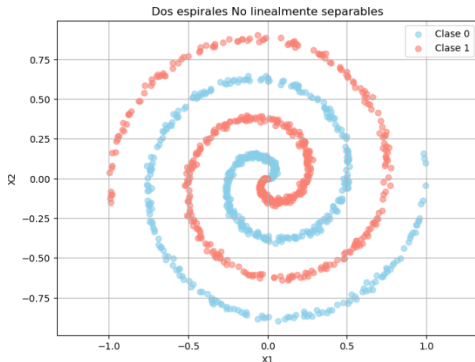
Redes Neuronales Multicapa (MLP)

Ejemplo: Clasificación de funciones No lineales con el perceptrón.



Redes Neuronales Multicapa (MLP)

Ejemplo: Clasificación de funciones No lineales con el perceptrón.



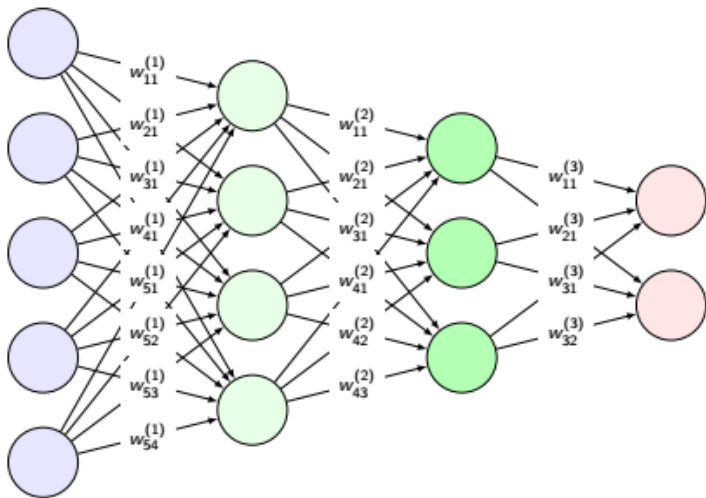
Red Neuronal Multicapa (MLP).

Es un modelo de aprendizaje automático compuesto por una capa de entrada, una o más capas ocultas con neuronas no lineales y una capa de salida, que permite aprender relaciones complejas entre datos.

- ▶ **Capa (Nodo) de entrada:** recibe los datos originales (características del problema).
- ▶ **Capas ocultas:** procesan la información mediante combinaciones lineales y funciones de activación no lineales, extrayendo representaciones cada vez más abstractas.
- ▶ **Capa de salida:** genera el resultado final, como una clase (clasificación) o un valor numérico (regresión), a partir de la representación interna aprendida.

Redes Neuronales Multicapa (MLP)

Arquitectura Típica de un MLP.



Redes Neuronales Multicapa (MLP)

La arquitectura más común en redes neuronales clásicas (MLP) es la de capas totalmente conectadas (**fully connected**).

En este tipo de capas:

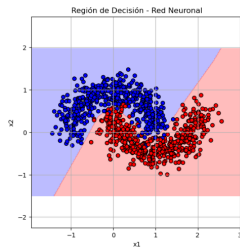
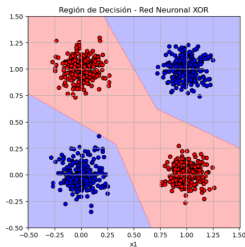
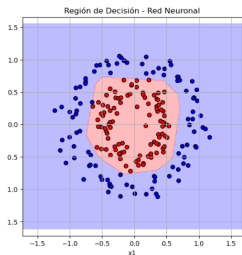
- ▶ Cada neurona de una capa se conecta con todas las neuronas de la siguiente capa.
- ▶ No existen conexiones entre neuronas de una misma capa.
- ▶ Cada neurona realiza una transformación no lineal:

$$a = \phi(\mathbf{w}^T \mathbf{x} + b)$$

donde ϕ , la función de activación, puede ser ReLU, Sigmoide, Tanh, etc.

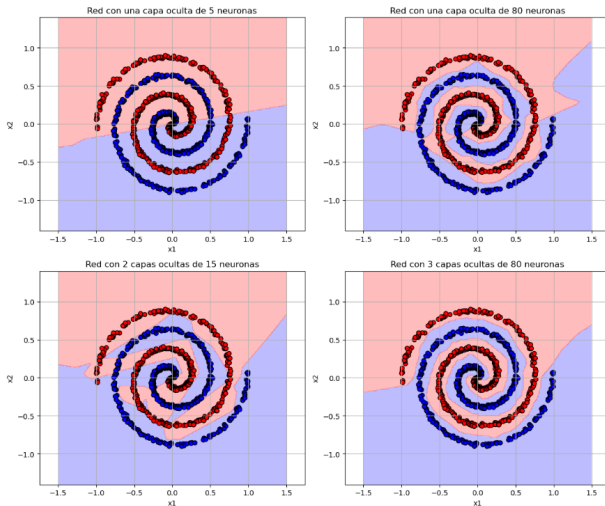
- ▶ La capa de entrada no son neuronas sino nodos (características).

Ejemplo: Clasificación de funciones No lineales con MLP.



Redes Neuronales Multicapa (MLP)

Ejemplo: Clasificación de funciones No lineales con MLP.
Variando el número de capas ocultas y de neuronas.



Ejemplo en Python: Redes Neuronales Multicapa (MLP).

Objetivo

Comprender el potencial de las redes neuronales en problemas No lineales.

► **Ver Notebook.**

Parámetros e Hiperparámetros en MLP

Parámetros

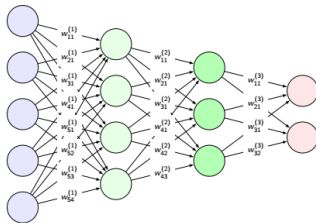
- ▶ Son los valores que el **modelo aprende automáticamente durante el entrenamiento** para minimizar la función de pérdida.
- ▶ En un MLP incluyen:
 - ▶ Pesos $W^{(l)}$ y Sesgos $b^{(l)}$

Hiperparámetros

- ▶ Son los valores que **se definen antes del entrenamiento** y controlan la arquitectura o el proceso de aprendizaje.
- ▶ No se aprenden directamente de los datos.
- ▶ Ejemplos en un MLP:
 - ▶ Número de capas y neuronas (Arquitectura)
 - ▶ Función de activación (Arquitectura)
 - ▶ Tasa de aprendizaje η (Entrenamiento)
 - ▶ Batch size, épocas, optimizador (Entrenamiento)

Redes Neuronales Multicapa (MLP)

- ▶ Un perceptrón multicapa (MLP - *Multilayer Perceptron*) es una red neuronal de tipo *feedforward*.
- ▶ Está compuesto por:
 - ▶ Una capa de entrada
 - ▶ Una o más capas ocultas
 - ▶ Una capa de salida
- ▶ Las capas están completamente conectadas.
- ▶ La salida de cada capa es la entrada de la siguiente.



- ▶ Cada neurona aplica una transformación lineal seguida de una función de activación no lineal:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \phi^{(l)}(z^{(l)})$$

- ▶ **Donde:**

- ▶ $z^{(l)}$: vector de entrada neta a la capa l
- ▶ $W^{(l)}$: matriz de pesos entre capas $l - 1$ y l
- ▶ $a^{(l-1)}$: vector de salidas (activaciones) de la capa anterior
- ▶ $b^{(l)}$: vector de sesgos de la capa l
- ▶ $\phi^{(l)}$: función de activación de la capa l
- ▶ $a^{(l)}$: activación (salida) de la capa l

Parámetros de un MLP.

- ▶ Se ajustan durante el entrenamiento por medio de algoritmos como retropropagación y optimización por descenso de gradiente.
 - ▶ **Pesos ($W^{(l)}$):**
 - ▶ Determinan la fuerza de conexión entre capas.
 - ▶ **Aprendidos mediante retropropagación.**
 - ▶ **Sesgos ($b^{(l)}$):**
 - ▶ Permiten el desplazamiento de las funciones de activación.
 - ▶ Ambos se actualizan mediante **descenso de gradiente**:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

Hiperparámetros de un MLP.

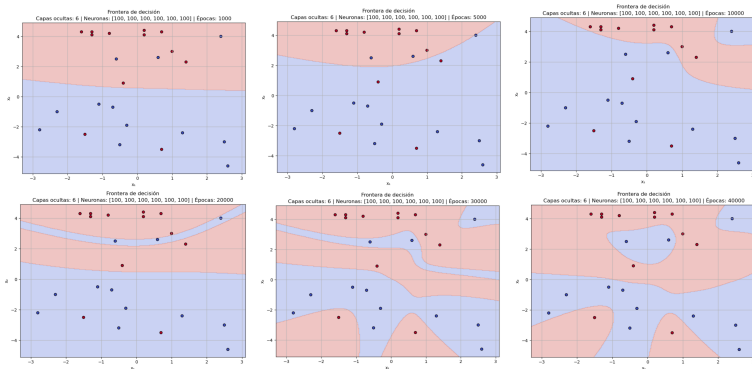
- ▶ Son **definidos manualmente** y pueden tener un gran impacto en el rendimiento.
- ▶ Se ajustan por validación cruzada o **prueba y error**.

Hiperparámetros principales:

- ▶ Número de capas ocultas y de neuronas por capa
- ▶ Función de activación (sigmoid, ReLU, tanh)
- ▶ Tasa de aprendizaje (η)
- ▶ Número de épocas
- ▶ Tamaño del batch
- ▶ **Regularización** (L2: $\lambda \|W\|^2$)
- ▶ Algoritmo de optimización (SGD, Adam, etc.)

Regularización.

Las redes neuronales con muchos parámetros pueden **sobreajustar** (*overfitting*) los datos de entrenamiento.



Ver código python.

Regularización.

- ▶ Es una técnica que **añade una penalización al valor de la función de pérdida** para evitar modelos demasiado complejos.
- ▶ Obliga a los pesos a mantenerse pequeños o dispersos, lo que favorece modelos más simples y que generalicen mejor.

Tipos comunes:

- ▶ **Regularización L_2 (Ridge)**: penaliza la suma de los cuadrados de los pesos:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{original}} + \lambda \sum_i w_i^2$$

- ▶ **Regularización L_1 (Lasso)**: penaliza la suma de los valores absolutos de los pesos:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{original}} + \lambda \sum_i |w_i|$$

Función de pérdida.

- ▶ La red MLP se entrena para minimizar una función de pérdida.
- ▶ En clasificación multiclase se usa normalmente la entropía cruzada:

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- ▶ Donde y_k es la etiqueta verdadera (one-hot) y $\hat{y}_k$ la predicción.
 - ▶ La codificación **one-hot** es una forma de representar una clase como un vector binario.

Codificación One-Hot de la Etiqueta Verdadera.

- ▶ Supone que hay K clases posibles, y cada clase se representa como un vector de dimensión K .
- ▶ La clase verdadera se indica con un **1** en la posición correspondiente, y **0** en las demás.

Ejemplo numérico: Si hay $K = 5$ clases y la clase verdadera es la número 3, entonces:

Etiqueta $y_k = 3 \Rightarrow$ Vector one-hot: $\mathbf{y} = [0, 0, 1, 0, 0]$

Importancia

Este vector se usa para comparar contra la salida del modelo (por ejemplo, en regresión softmax) al calcular la función de pérdida.

Resumen: Parámetros vs. Hiperparámetros.

Tipo	Nombre	Descripción
Parámetro	Pesos W	Conectan capas, se aprenden
Parámetro	Sesgos b	Desplazan activaciones
Hiperparámetro	Nº capas ocultas	Profundidad de la red
Hiperparámetro	Neuronas por capa	Capacidad del modelo
Hiperparámetro	Activación	Introduce no linealidad
Hiperparámetro	Optimizador	Método de actualización
Hiperparámetro	Learning rate η	Tamaño de actualización
Hiperparámetro	Regularización λ	Evita sobreajuste
Hiperparámetro	Épocas	Iteraciones de entrenamiento
Hiperparámetro	Batch size	Subconjunto por iteración

Capacidad de aproximación de los MLP.

- ▶ Un MLP con al menos una capa oculta puede aproximar cualquier función continua sobre un conjunto compacto.
- ▶ Este resultado se conoce como:

Teorema del aproximador universal

Una red MLP con función de activación no lineal y una capa oculta suficientemente grande puede aproximar cualquier función continua con precisión arbitraria.

- ▶ Aunque una sola capa oculta con suficientes neuronas puede aproximar cualquier función continua, **en la práctica se necesita una estructura profunda** para hacerlo eficientemente.

Número de capas ocultas y de neuronas por capa.

- ▶ No existe una fórmula exacta para la selección del número de capas ocultas y de neuronas por capa..
- ▶ La selección depende del problema y los datos.

Estrategias prácticas:

- ▶ **Número de capas ocultas:**
 - ▶ **1–2 capas:** suficiente para problemas simples o tabulares.
 - ▶ **2–4 capas:** útil para tareas más complejas como clasificación de imágenes simples.
 - ▶ **Redes profundas (5+ capas):** recomendadas para tareas muy complejas (visión, lenguaje, etc.).

Estrategias prácticas:

- ▶ **Número de neuronas por capa:**
 - ▶ Debe ser suficiente para captar la complejidad del problema, pero no tan alto como para sobreajustar.
 - ▶ Generalmente se prueba con números como 16, 32, 64, 128 ...
 - ▶ Puede usarse una estructura piramidal (decreciente o creciente) según el tipo de tarea.
- ▶ **Evaluación:** Se recomienda usar validación cruzada o una partición de validación para comparar arquitecturas.
- ▶ **Regularización y técnicas auxiliares:** Si se usan muchas capas o neuronas, es importante emplear técnicas como dropout, batch normalization o regularización L_1/L_2 .

Comenzar con una arquitectura pequeña y aumentar la complejidad solo si es necesario.

Funciones de Activación.

- ▶ Son componentes clave en las redes neuronales.
- ▶ Introducen **no linealidad** en el modelo.
- ▶ Sin activaciones no lineales, la red actúa como una transformación lineal, sin importar cuántas capas tenga.
- ▶ Permiten aprender relaciones complejas entre entrada y salida.

▶ Dada la salida lineal de una neurona: $z = \mathbf{w}^T \mathbf{x} + b$

▶ Se aplica una función de activación ϕ : $a = \phi(z)$

▶ Esto produce la **salida activada** de la neurona.

Propiedades Deseables.

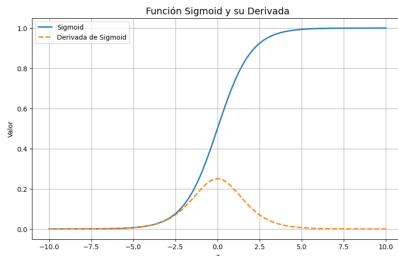
Una buena función de activación debe ser:

- ▶ **No lineal**
- ▶ **Derivable** (para el uso de retropropagación)
- ▶ **Computacionalmente eficiente**
- ▶ **Estable numéricamente** (evitar saturación o explosión de gradientes)

Función de Activación: Sigmoide.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

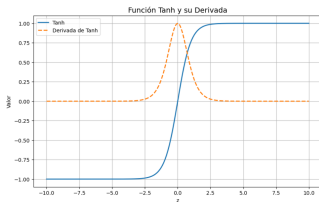
- ▶ Rango de salida: $(0, 1)$
- ▶ Se interpreta como probabilidad
- ▶ **Derivada máxima: 0.25**
- ▶ **Problemas:** saturación, gradiente desvanecido, no centrada en cero



Función de Activación: Tangente Hiperbólica (tanh).

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh'(x) = 1 - \tanh^2(x)$$

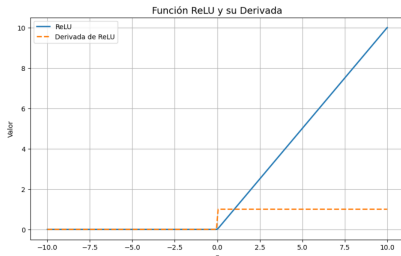
- ▶ Rango de salida: $(-1, 1)$
- ▶ Centrada en cero: más adecuada que sigmoide para muchas redes
- ▶ **Problemas:** saturación, gradiente desvanecido



Función de Activación: ReLU.

$$\text{ReLU}(x) = \max(0, x), \quad \text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

- ▶ Rango: $[0, \infty)$
- ▶ Simple y eficiente computacionalmente
- ▶ **Ventaja:** no sufre de gradiente desvanecido (cuando $x > 0$)
- ▶ **Problema:** *Dying ReLU*, neuronas que dejan de activarse



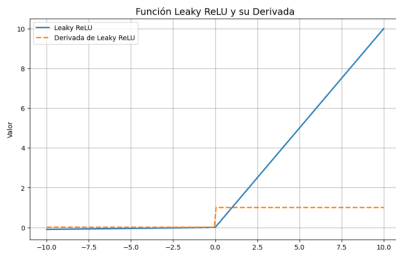
Problema: **Dying ReLU**.

- ▶ Una neurona ReLU muere cuando $x \leq 0$ de forma permanente:
 - ▶ Salida siempre cero
 - ▶ Derivada cero *Rightarrow* no aprende
- ▶ Causas:
 - ▶ Tasa de aprendizaje alta
 - ▶ Inicialización deficiente
- ▶ Soluciones:
 - ▶ Leaky ReLU
 - ▶ Tasa de aprendizaje más baja

Función de Activación: Leaky ReLU.

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0, \quad \alpha \in (0, 1) \end{cases}$$

- ▶ Variante de ReLU que permite pequeños gradientes cuando $x < 0$
- ▶ Ayuda a evitar el problema de neuronas muertas



Función de Activación: SELU (Scaled Exponential Linear Unit).

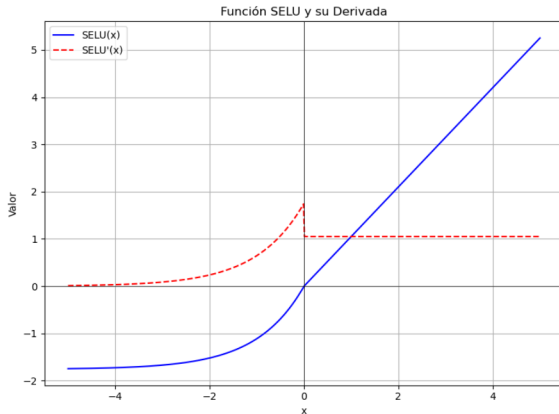
$$\text{SELU}(x) = \lambda \begin{cases} x & \text{si } x > 0 \\ \alpha(e^x - 1) & \text{si } x \leq 0 \end{cases}$$

$$\text{con } \lambda \approx 1,0507, \quad \alpha \approx 1,6733$$

- ▶ Mantiene las activaciones normalizadas (media ≈ 0 , varianza ≈ 1)
- ▶ Elimina la necesidad de Batch Normalization en redes profundas
- ▶ Requiere inicialización LeCun y AlphaDropout
- ▶ A diferencia de ReLU o Leaky ReLU, SELU asegura que incluso las neuronas con activaciones pequeñas o negativas contribuyan al aprendizaje, y que toda la red mantenga un flujo de datos estable.

Hiperparámetros: Funciones de Activación

Tiene una rama lineal para $x > 0$ y exponencial para $x < 0$.

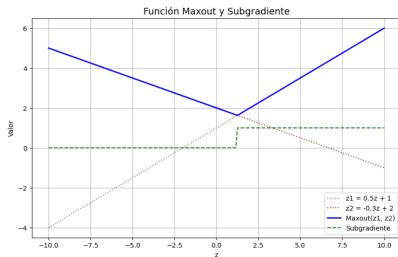


- SELU es útil especialmente en redes profundas completamente conectadas.

Función de Activación: Maxout.

$$\text{Maxout}(x) = \max(w_1^T x + b_1, w_2^T x + b_2)$$

- ▶ Generaliza ReLU y Leaky ReLU
- ▶ Aprende la función de activación durante el entrenamiento
- ▶ Mayor capacidad de representación, pero más costosa



Hiperparámetros: Funciones de Activación

- ▶ Para cada neurona Maxout, se utilizan varios pares de pesos y sesgos (w_i, b_i) .
- ▶ La salida de la neurona es el **máximo** entre todas las activaciones lineales:

$$\text{Maxout}(x) = \max_{i=1,\dots,k} (w_i^\top x + b_i)$$

- ▶ Maxout es una función de activación **adaptable**.
- ▶ Permite a la red **aprender la forma óptima de activación**, combinando múltiples funciones lineales.
- ▶ Puede **aproximar otras funciones de activación** si se le da suficiente capacidad.

Función Softmax.

- ▶ No es una función de activación en el sentido tradicional, pero sí se utiliza como función de activación en la **capa de salida** de redes neuronales para problemas de **clasificación multiclase**.

Dada una entrada $\mathbf{z} = [z_1, z_2, \dots, z_K]$, la función Softmax produce:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad \text{para } i = 1, \dots, K$$

Características clave:

- ▶ Se aplica sobre **todas las salidas simultáneamente** (función vectorial).
- ▶ Convierte salidas arbitrarias en **valores entre 0 y 1**, cuya **suma total es 1**.
- ▶ Es ideal para asignar **probabilidades a clases mutuamente excluyentes**.

Problema de Saturación.

- ▶ Una función de activación se dice **saturada** cuando su entrada cae en una región donde la salida cambia muy poco.
- ▶ En estas regiones, la derivada de la función es **cercana a cero**.
- ▶ Esto impide que se transmitan gradientes significativos durante el entrenamiento.

Saturación: Ejemplo con la función sigmoide.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- ▶ Para $x \ll 0$, $\sigma(x) \approx 0$, y $\sigma'(x) \approx 0$.
- ▶ Para $x \gg 0$, $\sigma(x) \approx 1$, y $\sigma'(x) \approx 0$.
- ▶ En ambos casos, la neurona entra en una región **saturada**.

Problema del Gradiente Desvanecido.

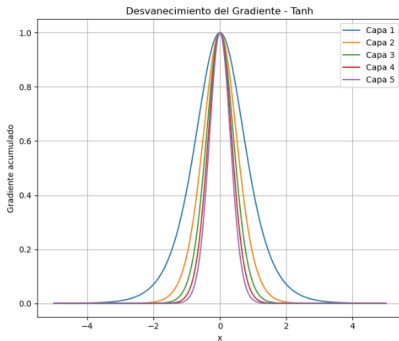
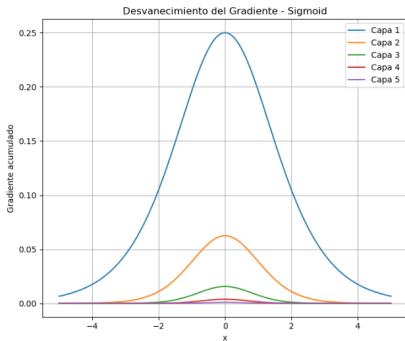
- ▶ Ocurre al aplicar retropropagación en redes profundas.
- ▶ Los gradientes **disminuyen exponencialmente** al propagarse hacia atrás.
- ▶ Es común con funciones cuya derivada es menor que 1, como sigmoid y tanh.

Sea la derivada total del error respecto a un peso en una capa profunda:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdot \dots \cdot \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial w}$$

- ▶ Si cada término $\frac{\partial a_i}{\partial a_{i-1}} \ll 1$, entonces: $\left| \frac{\partial \mathcal{L}}{\partial w} \right| \rightarrow 0$
- ▶ Esto ralentiza o detiene el aprendizaje en capas profundas.

Ejemplo del problema del Gradiente Desvanecido.



Consecuencias inmediatas:

- ▶ Las capas cercanas a la entrada reciben **gradientes extremadamente pequeños**.
- ▶ El **aprendizaje se vuelve ineficiente o se detiene por completo** en las primeras capas.
- ▶ **La red no logra aprender representaciones útiles en capas profundas**, por lo que se reduce la capacidad de aprendizaje y generalización de la red neuronal.

Soluciones Comunes al Gradiente Desvanecido.

Se han propuesto varias estrategias eficaces para mitigar este problema:

- ▶ **Funciones de activación alternativas:** ReLU, Leaky ReLU, ELU.
- ▶ **Inicialización adecuada de pesos:** técnicas como **Xavier** o **He**.
- ▶ **Normalización de capas:** uso de **Batch Normalization** para estabilizar las activaciones.
- ▶ **Algoritmos como ResNets** con conexiones residuales para facilitar el flujo de gradientes.

Estas estrategias ayudan a mantener gradientes significativos y permiten un entrenamiento más estable en redes profundas.

Problema de Explosión de Gradientes.

- ▶ Ocurre cuando los gradientes se vuelven **muy grandes** durante la retropropagación.
- ▶ Produce actualizaciones excesivas en los pesos, volviendo el entrenamiento **inestable** o **divergente**.
- ▶ Es común en redes profundas o recurrentes (RNNs).

Origen Matemático.

- ▶ Durante el entrenamiento de redes neuronales profundas, el gradiente del error se propaga desde las capas de salida hacia las capas anteriores utilizando la **regla de la cadena**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \prod_{k=l}^{L-1} \frac{\partial \mathbf{a}^{(k+1)}}{\partial \mathbf{z}^{(k+1)}} \cdot \frac{\partial \mathbf{z}^{(k+1)}}{\partial \mathbf{a}^{(k)}}$$

- ▶ En cada capa, se multiplica por:
 - ▶ La **derivada de la función de activación** $f'(\mathbf{z})$,
 - ▶ Y los **pesos** $\mathbf{W}$ de la capa siguiente.

- ▶ Por tanto, el gradiente en las primeras capas puede tomar la forma:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \propto \prod_{k=l}^{L-1} \mathbf{W}^{(k)} f'(\mathbf{z}^{(k)})$$

- ▶ Si las matrices $\mathbf{W}^{(k)}$ tienen valores grandes o la derivada $f'(\cdot)$ es mayor que 1 en alguna región, este producto puede **crecer exponencialmente** con el número de capas.
- ▶ Si muchas derivadas son **mayores que 1**, su producto puede crecer exponencialmente.
- ▶ Esto genera la **explosión de gradientes**.

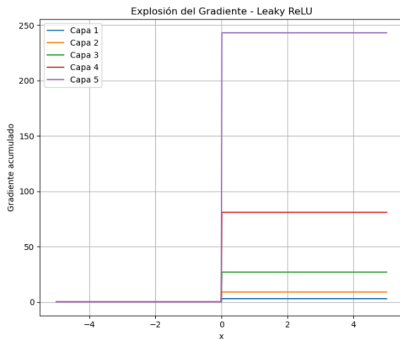
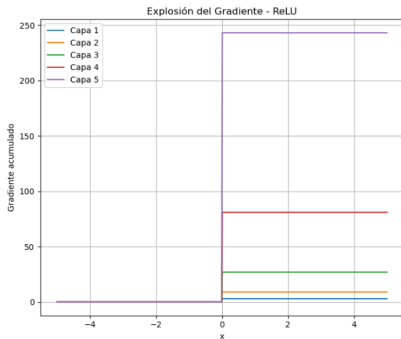
Hiperparámetros: Funciones de Activación

Las funciones de activación más propensas a causar o amplificar la **explosión de gradientes** son aquellas que:

- ▶ No están **acotadas por arriba** (pueden producir valores muy grandes).
- ▶ Tienen **derivadas mayores que 1** en parte de su dominio.
- ▶ Se combinan con **mala inicialización** o redes muy profundas.

Función	Motivo del riesgo
ReLU	No acotada por arriba: si $x \gg 0$, entonces $\text{ReLU}(x) = x$. Las activaciones pueden crecer sin límite.
Leaky ReLU	Igual que ReLU para $x > 0$; tampoco no acotada por arriba.
Tanh	Está acotada, pero su derivada puede amplificar valores cercanos a cero si los pesos no están bien inicializados. Riesgo en redes profundas.

Ejemplo del problema de Explosión del Gradiente.



Consecuencias de la Explosión de Gradientes.

- ▶ Pesos con valores extremadamente grandes.
- ▶ Función de pérdida que se vuelve indefinida o NaN.
- ▶ Entrenamiento inestable o divergente.
- ▶ Gradientes que oscilan violentamente.

Soluciones comunes

- ▶ **Inicialización adecuada:** He o Xavier.
- ▶ **Normalización de activaciones:** Batch Normalization, Layer Norm.
- ▶ Regularización o recorte de gradientes (gradient clipping): limitar su magnitud.

Ver código python.

Inicialización de pesos w y sesgos b .

Importancia de la inicialización.

- ▶ Una mala inicialización puede provocar:
 - ▶ **Gradiente desvanecido o explosión de gradientes**
 - ▶ Convergencia lenta o inestable
 - ▶ Atrapamiento en simetrías (si todos los pesos son iguales)
- ▶ La inicialización debe permitir:
 - ▶ Propagación eficiente del gradiente hacia atrás
 - ▶ Valores razonables de activaciones y derivadas

Hiperparámetros: Inicialización de pesos w y sesgos b .

Pesos W

- ▶ Se deben inicializar aleatoriamente para romper la simetría.
- ▶ La distribución depende del número de entradas y de la función de activación.

Sesgos b

- ▶ Suelen inicializarse en cero o con pequeños valores constantes.
- ▶ No afectan la simetría en la red.

Técnicas Clásicas de Inicialización de Pesos.

Técnica	Distribución	Recomendada para
Inicialización aleatoria	$\mathcal{N}(0, \epsilon)$	General (en desuso)
Inicialización cero	$w = 0$	Nunca usar
Glorot (Xavier)	$\mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$	tanh, sigmoid
He	$\mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$	ReLU, Leaky ReLU
LeCun	$\mathcal{N}\left(0, \frac{1}{n_{\text{in}}}\right)$	SELU
Ortogonal	Matriz ortogonal	RNNs y redes profundas

- ▶ $\mathcal{N}(\mu, \sigma^2)$: Distribución normal con media μ y varianza σ^2 .
- ▶ n_{in} : Número de neuronas de entrada a la capa actual.
- ▶ n_{out} : Número de neuronas en la capa actual.
- ▶ ϵ : Valor pequeño positivo usado en algunas inicializaciones antiguas.

Hiperparámetros: Inicialización de pesos w y sesgos b .

Inicialización Xavier (Glorot).

Mantener la varianza constante entre capas:

$$w_{ij} \sim \mathcal{U} \left(-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}} \right)$$

O con distribución normal:

$$w_{ij} \sim \mathcal{N} \left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}} \right)$$

- ▶ n_{in} : Es el número de unidades de entrada a una capa determinada. Es decir, la cantidad de neuronas en la capa anterior.
- ▶ n_{out} : Es el número de unidades de salida de la capa. Es decir, la cantidad de neuronas en la capa actual que se está inicializando.

Usar con: funciones simétricas como $\tanh$, sigmoid .

Hiperparámetros: Inicialización de pesos w y sesgos b .

Inicialización He.

Compensar el efecto de ReLU (que descarta valores negativos):

$$w_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \quad \text{o} \quad w_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right)$$

Usar con: ReLU, Leaky ReLU, ELU.

Hiperparámetros: Inicialización de pesos w y sesgos b .

Inicialización LeCun.

Preservar la varianza de las activaciones con funciones como sigmoid o SELU:

$$w_{ij} \sim \mathcal{N}\left(0, \frac{1}{n_{\text{in}}}\right) \quad \text{o} \quad w_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{3}{n_{\text{in}}}}, \sqrt{\frac{3}{n_{\text{in}}}}\right)$$

Usar con: Sigmoid, SELU, funciones suaves no centradas en cero.

Hiperparámetros: Inicialización de pesos w y sesgos b .

Inicialización Ortogonal.

Genera una matriz de pesos ortogonal para preservar la norma del flujo de información a través de las capas.

Si $W \in \mathbb{R}^{m \times n}$ es la matriz de pesos, se construye a partir de una matriz ortogonal Q mediante descomposición:

$$W = Q \quad \text{tal que} \quad Q^T Q = I$$

Si $m \neq n$, se toma una submatriz de una matriz ortogonal cuadrada generada por descomposición QR.

- ▶ Mantiene la varianza del gradiente estable en redes profundas.
- ▶ Reduce el problema del desvanecimiento/explosión del gradiente.
- ▶ Muy útil en RNNs, LSTM, redes profundas con muchas capas ocultas.

Inicialización de Sesgos.

- ▶ Los sesgos b se suelen inicializar como:

$$b_i = 0 \quad \text{o} \quad b_i \sim \mathcal{N}(0, \epsilon)$$

- ▶ Si se usa ReLU, puede ser útil usar valores pequeños positivos:

$$b_i = 0,01$$

- ▶ Evitar valores grandes que desbalancen las activaciones.

Resumen de Recomendaciones.

- ▶ Evitar inicializaciones con ceros en los pesos.
- ▶ Usar:
 - ▶ **He** para ReLU y variantes.
 - ▶ **Xavier/Glorot** para funciones simétricas.
 - ▶ **LeCun** para SELU.
- ▶ Inicializar sesgos en cero o con pequeños valores positivos.
- ▶ Para redes profundas o RNNs: considerar inicialización ortogonal.

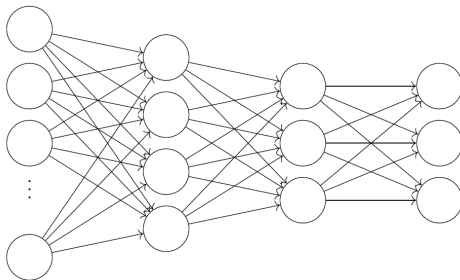
Ver código python.

Hiperparámetros: Inicialización de pesos w y sesgos b .

Propagacion hacia adelante.

Sea considera una red neuronal con la siguiente arquitectura:

- ▶ N neuronas en la capa de entrada
- ▶ Dos capas ocultas con H_1 y H_2 neuronas respectivamente
- ▶ 3 neuronas en la capa de salida



Propagación hacia Adelante: Primera Capa Oculta.

- ▶ Sea $\mathbf{x} \in \mathbb{R}^N$ el vector de entrada.
- ▶ La activación de la primera capa oculta es:

$$\mathbf{z}^{[1]} = W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}, \quad \mathbf{a}^{[1]} = \phi^{[1]}(\mathbf{z}^{[1]})$$

donde:

- ▶ $W^{[1]} \in \mathbb{R}^{H_1 \times N}$: pesos entre la entrada y la capa oculta 1
- ▶ $\mathbf{b}^{[1]} \in \mathbb{R}^{H_1}$: sesgos
- ▶ $\phi^{[1]}$: función de activación (e.g. ReLU)

Segunda Capa Oculta.

- ▶ La salida de la segunda capa oculta se obtiene aplicando:

$$\mathbf{z}^{[2]} = W^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}, \quad \mathbf{a}^{[2]} = \phi^{[2]}(\mathbf{z}^{[2]})$$

donde:

- ▶ $W^{[2]} \in \mathbb{R}^{H_2 \times H_1}$: pesos de la capa oculta 1 a la capa oculta 2
- ▶ $\mathbf{b}^{[2]} \in \mathbb{R}^{H_2}$: sesgos
- ▶ $\phi^{[2]}$: función de activación (e.g. ReLU)

Capa de Salida.

- ▶ La salida de la red se obtiene como:

$$\mathbf{z}^{[3]} = W^{[3]}\mathbf{a}^{[2]} + \mathbf{b}^{[3]}, \quad \hat{\mathbf{y}} = \phi^{[3]}(\mathbf{z}^{[3]})$$

donde:

- ▶ $W^{[3]} \in \mathbb{R}^{3 \times H_2}$: pesos hacia la capa de salida
- ▶ $\mathbf{b}^{[3]} \in \mathbb{R}^3$: sesgos
- ▶ $\phi^{[3]}$: función de activación de salida (e.g. softmax)

Ecuación General del Flujo hacia Adelante

La red neuronal aplica una composición de transformaciones lineales y funciones de activación:

$$\hat{\mathbf{y}} = \phi^{[3]} \left(W^{[3]} \cdot \phi^{[2]} \left(W^{[2]} \cdot \phi^{[1]} \left(W^{[1]} \cdot \mathbf{x} + \mathbf{b}^{[1]} \right) + \mathbf{b}^{[2]} \right) + \mathbf{b}^{[3]} \right)$$

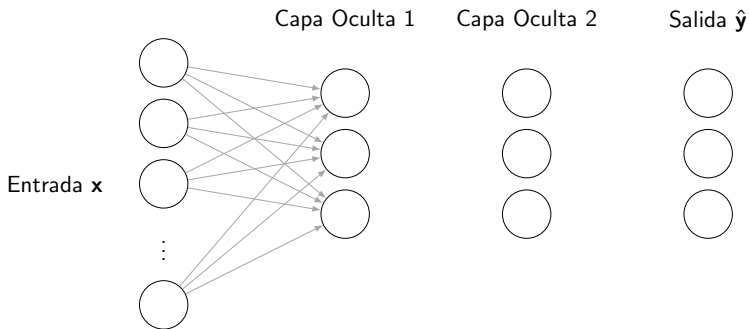
Interpretación:

- ▶ Cada capa aplica una transformación afín seguida de una activación no lineal.
- ▶ El proceso se puede ver como una función compuesta:

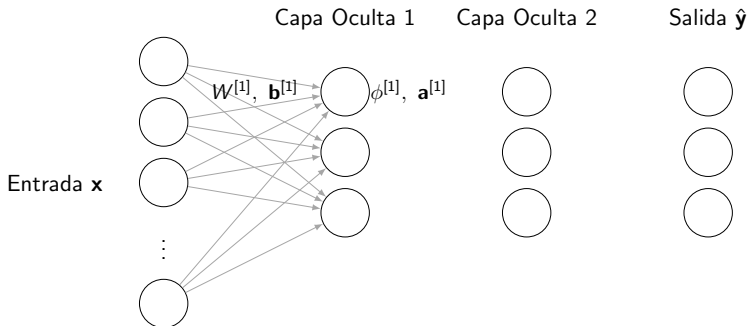
$$\hat{\mathbf{y}} = f^{[3]} \circ f^{[2]} \circ f^{[1]}(\mathbf{x})$$

donde cada $f^{[\ell]}(\cdot) = \phi^{[\ell]}(W^{[\ell]} \cdot + \mathbf{b}^{[\ell]})$.

Propagación hacia Adelante:.

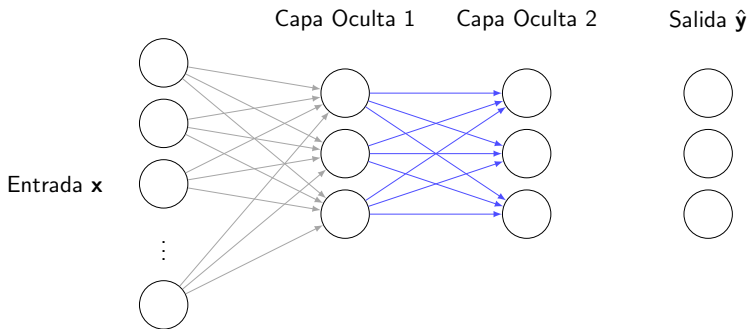


Propagación hacia Adelante:.

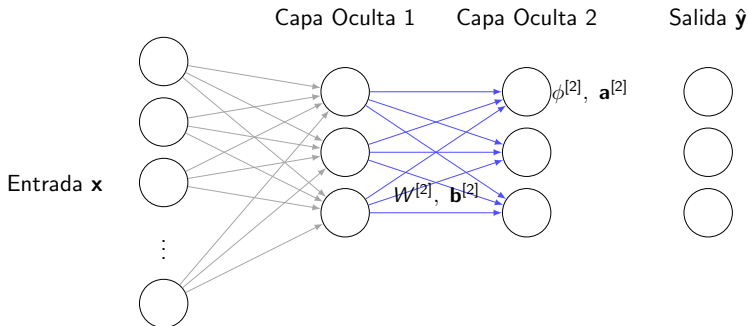


Capa Oculta 1: $\mathbf{a}^{[1]} = \phi^{[1]}(W^{[1]}\mathbf{x} + \mathbf{b}^{[1]})$

Propagación hacia Adelante:.

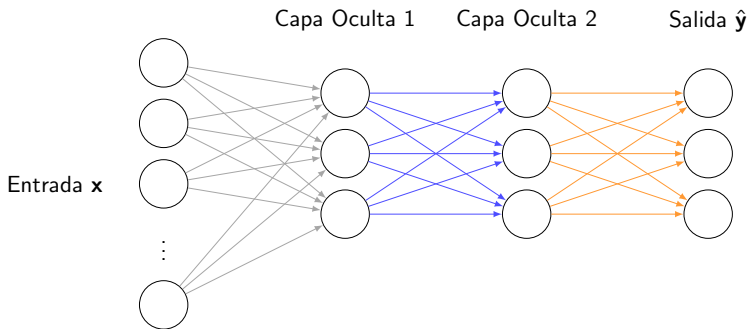


Propagación hacia Adelante:.



Capa Oculta 2: $\mathbf{a}^{[2]} = \phi^{[2]}(W^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]})$

Propagación hacia Adelante:.



Funciones de Pérdida (costo).

- ▶ La **función de pérdida** mide cuán bien la red está haciendo sus predicciones.
- ▶ Cuantifica el **error** entre la salida real (**y**) y la salida predicha por la red (**$\hat{y}$**).
 - ▶ **y** representa la etiqueta verdadera o deseada,
 - ▶ **$\hat{y}$** es la salida generada por la red.
- ▶ El proceso de **retropropagación** busca minimizar esta función ajustando los pesos del modelo.

Error Absoluto Medio (MAE).

$$\mathcal{L}_{\text{MAE}}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{k=1}^K |\hat{y}_k - y_k|$$

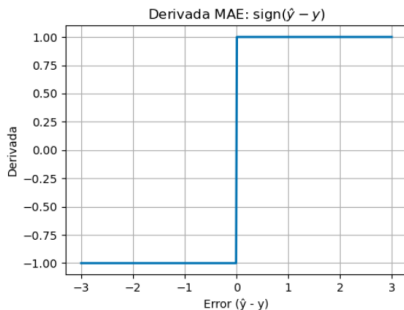
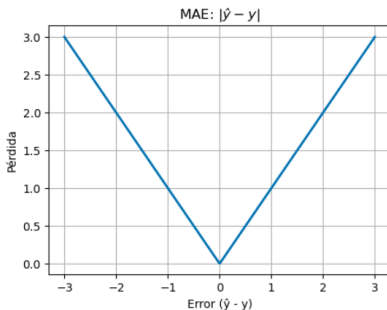
Derivada:

$$\frac{\partial \mathcal{L}_{\text{MAE}}}{\partial \hat{y}_k} = \text{sign}(\hat{y}_k - y_k)$$

- ▶ **Uso:** Problemas de regresión donde se desea robustez frente a valores atípicos.
- ▶ **Ventaja:** No amplifica outliers como MSE.
- ▶ **Desventaja:** Derivada no continua en cero, lo que puede dificultar la optimización.

Hiperparámetros: Funciones de Pérdida (Costo).

Función de pérdida: Error Absoluto Medio y su derivada.



Error Cuadrático Medio (MSE).

$$\mathcal{L}_{\text{MSE}}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \sum_{k=1}^K (\hat{y}_k - y_k)^2$$

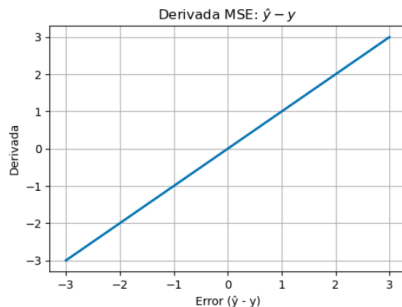
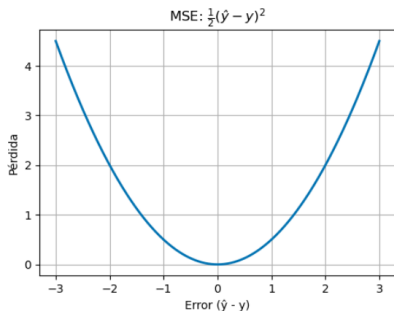
Derivada:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}_k} = \hat{y}_k - y_k$$

- ▶ **Uso:** Problemas de regresión.
- ▶ **Ventaja:** Suave, fácil de derivar.
- ▶ **Desventaja:** Muy sensible a valores atípicos (outliers).

Hiperparámetros: Funciones de Pérdida (Costo).

Función de pérdida: Error Cuadrático Medio y su derivada.



Entropía Cruzada para Clasificación Multiclase.

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

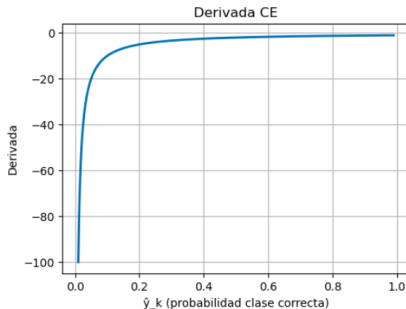
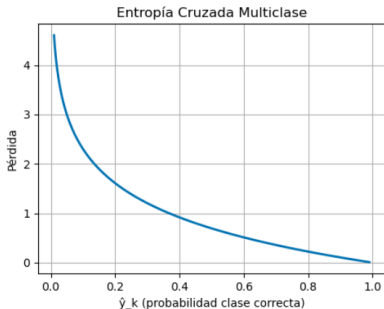
Derivada:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial \hat{y}_k} = - \frac{y_k}{\hat{y}_k}$$

- ▶ **Uso:** Clasificación multiclase con softmax.
- ▶ **Ventaja:** Muy efectiva para modelos probabilísticos.
- ▶ **Precaución:** $\hat{y}_k$ debe estar en $(0, 1)$, se recomienda suavizar.

Hiperparámetros: Funciones de Pérdida (Costo).

Función de pérdida: Entropía Cruzada para Clasificación Multiclase y su derivada.



Binary Cross-Entropy.

$$\mathcal{L}_{\text{BCE}}(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

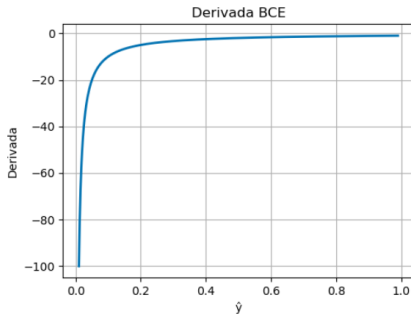
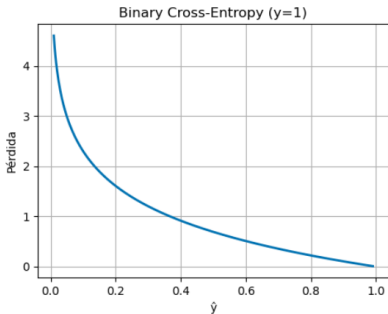
Derivada:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial \hat{y}} = - \left(\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} \right)$$

- ▶ **Uso:** Clasificación binaria (salida con sigmoid).
- ▶ **Derivada:** Relacionada con la verosimilitud de Bernoulli.
- ▶ **Recomendación:** Controlar $\hat{y} = 0$ o 1 con *clipping*.

Hiperparámetros: Funciones de Pérdida (Costo).

Función de pérdida: Binary Cross-Entropy y su derivada.



Hinge Loss (SVM).

$$\mathcal{L}_{\text{hinge}}(\hat{y}, y) = \max(0, 1 - y \cdot \hat{y})$$

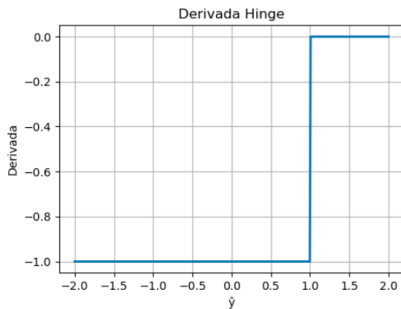
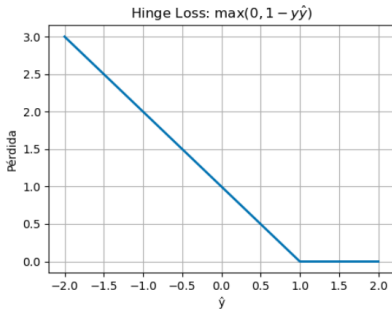
Derivada:

$$\frac{\partial \mathcal{L}_{\text{hinge}}}{\partial \hat{y}} = \begin{cases} -y & \text{si } 1 - y \cdot \hat{y} > 0 \\ 0 & \text{en otro caso} \end{cases}$$

- ▶ **Uso:** Clasificación binaria con etiquetas $y \in \{-1, 1\}$.
- ▶ **Inspirado en:** Margen de clasificadores SVM.
- ▶ **No común** en redes modernas, pero importante históricamente.

Hiperparámetros: Funciones de Pérdida (Costo).

Función de pérdida: Hinge Loss y su derivada.



Huber Loss: Mezcla MSE y MAE.

$$\mathcal{L}_{\delta}(\hat{y}, y) = \begin{cases} \frac{1}{2}(\hat{y} - y)^2 & \text{si } |\hat{y} - y| < \delta \\ \delta \cdot (|\hat{y} - y| - \frac{1}{2}\delta) & \text{si } |\hat{y} - y| \geq \delta \end{cases}$$

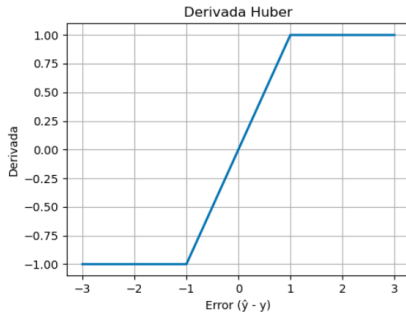
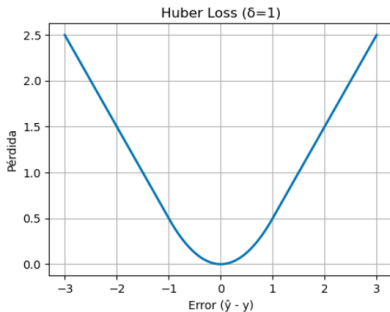
Derivada:

$$\frac{\partial \mathcal{L}_{\delta}}{\partial \hat{y}} = \begin{cases} \hat{y} - y & \text{si } |\hat{y} - y| < \delta \\ \delta \cdot \text{sign}(\hat{y} - y) & \text{si } |\hat{y} - y| \geq \delta \end{cases}$$

- ▶ **Uso:** Regresión robusta ante outliers.
- ▶ **Controla transición** entre MSE y MAE con δ .

Hiperparámetros: Funciones de Pérdida (Costo).

Función de pérdida: Huber Loss y su derivada.



Términos en la Función de Pérdida.

La función de pérdida total suele componerse de varios términos:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{datos}} + \lambda_1 \cdot \mathcal{L}_{\text{regularización}} + \lambda_2 \cdot \mathcal{L}_{\text{otros}}$$

- ▶ $\mathcal{L}_{\text{datos}}$: mide el error de predicción (mencionadas anteriormente, por ejemplo, entropía cruzada).
- ▶ $\mathcal{L}_{\text{regularización}}$: penaliza pesos grandes para evitar sobreajuste.
- ▶ $\mathcal{L}_{\text{otros}}$: otros términos según la tarea (adversarios, reconstrucción, etc.).

Regularización.

- ▶ La regularización controla la complejidad del modelo, previniendo el sobreajuste.
- ▶ Se agrega un término a la función de pérdida que penaliza los pesos grandes.
- ▶ Permite una mejor generalización a nuevos datos.

Función de pérdida con regularización:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{datos}} + \text{término de regularización}$$

Regularización L2 (Ridge).

$$\mathcal{L}_{L2} = \lambda \sum_{i,j} W_{ij}^2$$

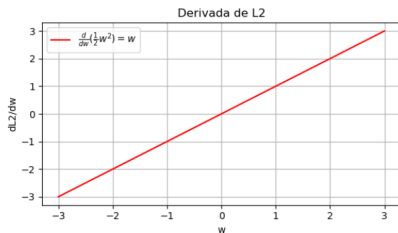
Derivada.

$$\frac{\partial \mathcal{L}_{L2}}{\partial W_{ij}} = 2\lambda W_{ij}$$

- ▶ Penaliza pesos grandes sin forzarlos a cero.
- ▶ Suaviza la solución, distribuyendo los valores de los pesos.
- ▶ Común en redes profundas con muchas conexiones.
- ▶ **Ventaja:** Estable y fácil de derivar.
- ▶ **Desventaja:** No realiza selección de características.

Hiperparámetros: Funciones de Pérdida (Costo).

Función de regularización $\mathcal{L}_{L2}$.



Regularización L1 (Lasso).

$$\mathcal{L}_{L1} = \lambda \sum_{i,j} |W_{ij}|$$

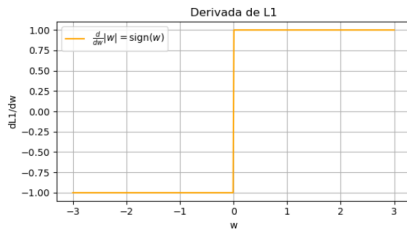
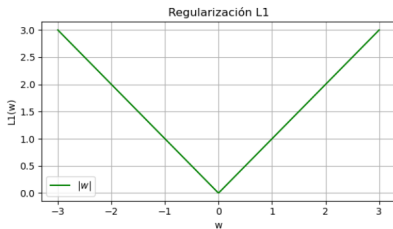
Derivada (subgradiente).

$$\frac{\partial \mathcal{L}_{L1}}{\partial W_{ij}} = \begin{cases} \lambda & \text{si } W_{ij} > 0 \\ -\lambda & \text{si } W_{ij} < 0 \\ \text{indefinido (usar subgradiente)} & \text{si } W_{ij} = 0 \end{cases}$$

- ▶ Induce esparsidad: algunos pesos se vuelven exactamente cero.
- ▶ Útil cuando se espera que solo unas pocas conexiones sean relevantes.
- ▶ **Ventaja:** Realiza selección de características.
- ▶ **Desventaja:** No siempre es diferenciable, lo cual complica la optimización.

Hiperparámetros: Funciones de Pérdida (Costo).

Función de regularización $\mathcal{L}_{L1}$.



Regularización Elástica (Elastic Net).

$$\mathcal{L}_{\text{elástica}} = \alpha \mathcal{L}_{L1} + (1 - \alpha) \mathcal{L}_{L2}$$

Derivada.

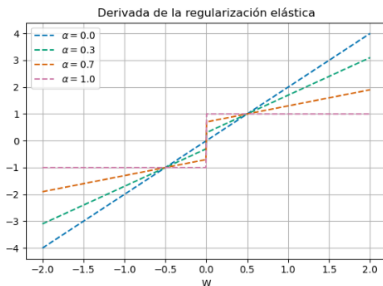
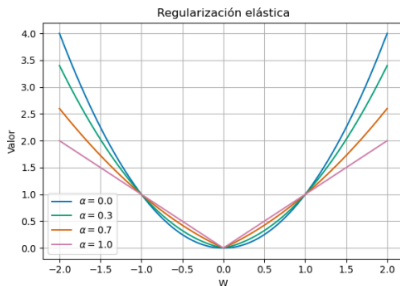
$$\frac{\partial \mathcal{L}_{\text{elástica}}}{\partial W_{ij}} = \alpha \frac{\partial \mathcal{L}_{L1}}{\partial W_{ij}} + (1 - \alpha) \frac{\partial \mathcal{L}_{L2}}{\partial W_{ij}}$$

- ▶ Combina las ventajas de L1 (Lasso) y L2 (Ridge).
- ▶ Promueve la selección de características y estabilidad en los pesos.
- ▶ Útil cuando existen muchas variables correlacionadas.
- ▶ **Ventaja:** Controla la magnitud de los pesos y realiza selección automática.
- ▶ **Desventaja:** Parámetro adicional ($\alpha \in [0, 1]$).

Hiperparámetros: Funciones de Pérdida (Costo).

Función de regularización **Elástica**.

- ▶ $\alpha = 1$: solo L1.
- ▶ $\alpha = 0$: solo L2.



Componentes adicionales de la función de pérdida: $\mathcal{L}_{\text{otros}}$

- ▶ Además de la función de pérdida de los datos y la regularización de los pesos, **se pueden incluir otras penalizaciones o restricciones**, dependiendo del objetivo del modelo.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{datos}} + \lambda_1 \mathcal{L}_{\text{reg}} + \lambda_2 \mathcal{L}_{\text{otros}}$$

- ▶ **Propósito:** Equilibrar precisión del modelo, capacidad de generalización y comportamiento específico deseado.

Técnicas de Optimización.

- ▶ La optimización busca minimizar la función de pérdida total

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{datos}}(\theta) + \mathcal{L}_{\text{reg}}(\theta)$$

donde:

- ▶ $\mathcal{L}_{\text{datos}}$: mide el error en las predicciones.
- ▶ $\mathcal{L}_{\text{reg}}$: controla la complejidad del modelo (por ejemplo, L2).
- ▶ Se ajustan los pesos θ para minimizar $\mathcal{L}$ mediante técnicas de optimización.

Descenso del Gradiente.

- ▶ Método de optimización clásico que calcula el gradiente de la función de pérdida **usando todos los datos del conjunto de entrenamiento**.
- ▶ Produce actualizaciones de pesos estables y precisas, pero requiere mucho cómputo por iteración.

Actualización de pesos:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta)$$

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ $\mathcal{L}(\theta)$: función de pérdida total calculada sobre todo el conjunto de entrenamiento.

Ventajas:

- ▶ Proporciona un descenso suave y consistente hacia el mínimo.
- ▶ Facilita el análisis teórico de la convergencia.
- ▶ Menor varianza en la dirección del gradiente.

Desventajas:

- ▶ Muy costoso computacionalmente para conjuntos de datos grandes.
- ▶ No se puede comenzar la actualización hasta haber procesado todos los datos.
- ▶ Puede quedar atrapado en mínimos locales en problemas no convexos.

Uso recomendado:

- ▶ Con conjuntos de datos pequeños o medianos.
- ▶ En tareas donde la precisión y estabilidad del descenso es prioritaria.

Gradiente Descendente Estocástico (SGD).

- ▶ Variante del descenso del gradiente que actualiza los pesos **utilizando una sola muestra aleatoria** del conjunto de entrenamiento.
- ▶ Introduce ruido en el proceso de optimización, lo cual puede ayudar a escapar de mínimos locales.

Actualización de pesos:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(x^{(i)}, \theta)$$

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ $x^{(i)}$: muestra individual seleccionada aleatoriamente del conjunto de entrenamiento.
- ▶ $\mathcal{L}(x^{(i)}, \theta)$: pérdida calculada con respecto a esa muestra.

Ventajas:

- ▶ Permite actualizaciones frecuentes con bajo costo computacional.
- ▶ Puede escapar de mínimos locales debido a su naturaleza ruidosa.
- ▶ Escala mejor a grandes conjuntos de datos que el descenso por lote completo.

Desventajas:

- ▶ Las trayectorias de optimización son inestables y ruidosas.
- ▶ Puede requerir más épocas para converger.
- ▶ Riesgo de oscilación en torno al mínimo óptimo.

Uso recomendado:

- ▶ Cuando se requiere una rápida aproximación en escenarios con datos masivos.
- ▶ Cuando se busca evitar mínimos locales poco profundos.

Mini-batch Gradient Descent.

- ▶ Variante de descenso del gradiente que calcula el gradiente **utilizando un subconjunto aleatorio del conjunto de entrenamiento (mini-lote)**.
- ▶ Combina las ventajas del descenso por lote completo y del gradiente descendente estocástico.

Actualización de pesos:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}_{\text{mini-batch}}(\theta)$$

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ Tamaño del mini-lote: número de ejemplos utilizados por iteración, típicamente entre 16 y 256.
- ▶ $\mathcal{L}_{\text{mini-batch}}(\theta)$: pérdida promedio sobre el mini-lote.

Ventajas:

- ▶ Menor varianza en las actualizaciones en comparación con SGD puro.
- ▶ Mejor eficiencia computacional en paralelo (GPU/TPU).
- ▶ Más rápida convergencia que el descenso por lote completo en conjuntos grandes.

Desventajas:

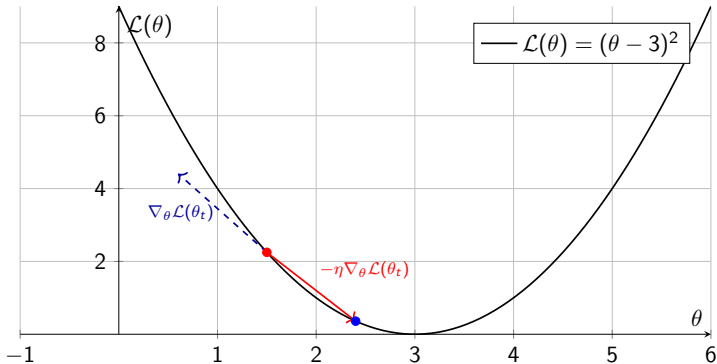
- ▶ La elección del tamaño del mini-lote afecta el rendimiento y la estabilidad.
- ▶ Aún puede haber oscilaciones si el mini-lote no es representativo.

Uso recomendado:

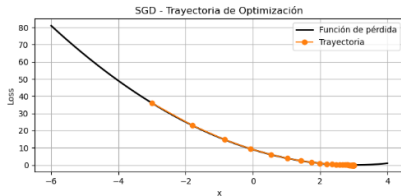
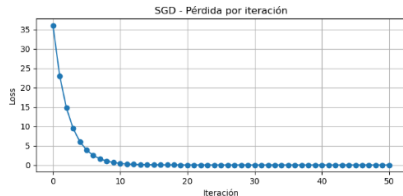
- ▶ **Es la técnica más usada actualmente** en redes neuronales profundas.
- ▶ Ideal para entrenamiento en conjuntos de datos grandes.

Ilustración de la actualización de los parámetros θ :

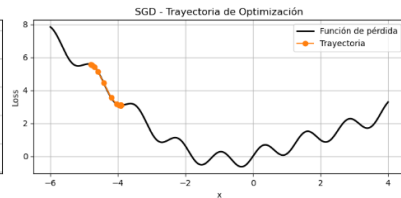
$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta)$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



Momentum.

- ▶ Extensión de SGD que introduce una variable de velocidad acumulada para suavizar las actualizaciones.
- ▶ Esta técnica ayuda a acelerar el descenso en direcciones coherentes y reduce la oscilación en dimensiones ruidosas.

Actualización de pesos:

$$\begin{aligned}v_t &\leftarrow \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta) \\ \theta &\leftarrow \theta - v_t\end{aligned}$$

En la primera iteración ($t = 1$) se hace $v_0 = 0$.

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ γ : coeficiente de momentum ($0 < \gamma < 1$), típicamente entre 0.9 y 0.99.

Ventajas:

- ▶ Acelera la convergencia en valles largos y estrechos.
- ▶ Reduce las oscilaciones en direcciones con gradientes ruidosos.
- ▶ Mejora la estabilidad frente a fluctuaciones del gradiente.

Desventajas:

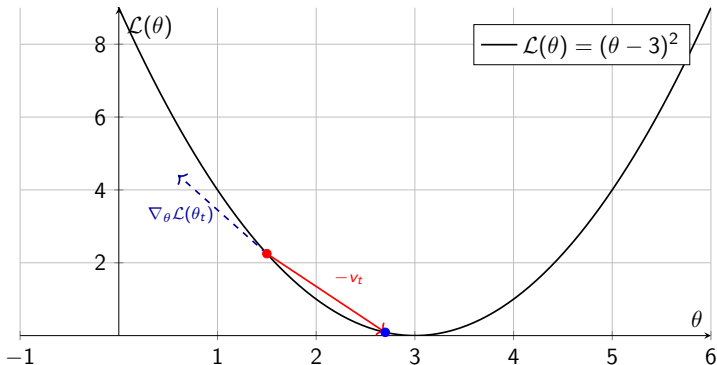
- ▶ Introduce un nuevo hiperparámetro γ que requiere ajuste.
- ▶ Puede sobrepasar el mínimo si γ o η son muy grandes.

Uso recomendado:

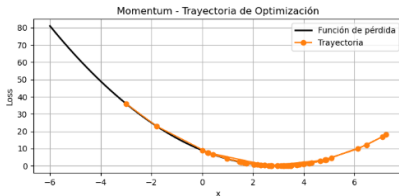
- ▶ Cuando se busca convergencia más rápida y estable que con SGD puro.
- ▶ Común en redes profundas con topologías complejas.

Ilustración de la actualización con Momentum:

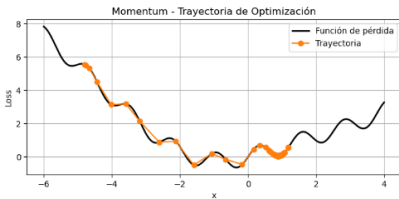
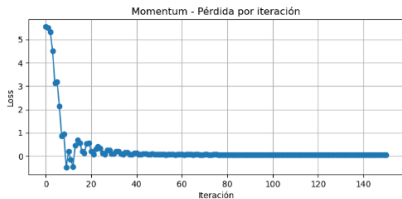
$$\begin{aligned}v_t &\leftarrow \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta_t) \\ \theta_{t+1} &\leftarrow \theta_t - v_t\end{aligned}$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



Nesterov Accelerated Gradient (NAG).

- ▶ Variante mejorada del método Momentum que calcula el gradiente en la posición anticipada del parámetro.
- ▶ Proporciona una “corrección anticipada” que puede mejorar la precisión de la dirección de descenso.

Actualización de pesos:

$$\begin{aligned}v_t &\leftarrow \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta - \gamma v_{t-1}) \\ \theta &\leftarrow \theta - v_t\end{aligned}$$

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ γ : coeficiente de momentum (típicamente entre 0.9 y 0.99).

Ventajas:

- ▶ Mejora la dirección de descenso gracias al uso de gradientes anticipados.
- ▶ Proporciona convergencia más precisa y rápida que el Momentum clásico.
- ▶ Reduce aún más la oscilación cerca del mínimo.

Desventajas:

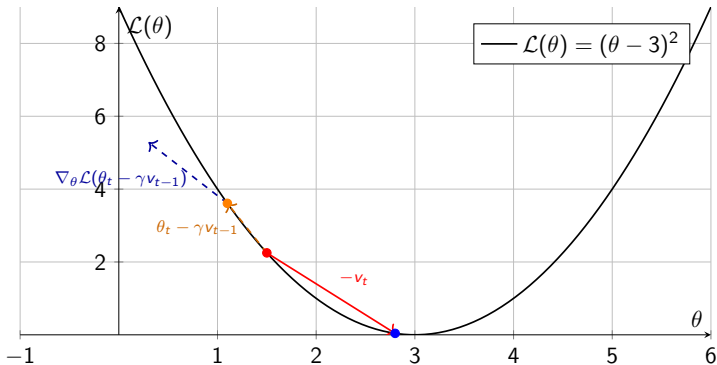
- ▶ Computacionalmente más costoso por evaluar el gradiente en una posición modificada.
- ▶ Aumenta la complejidad de implementación.

Uso recomendado:

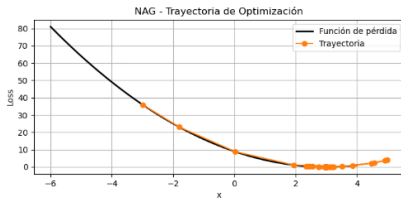
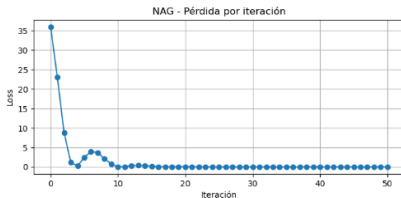
- ▶ Cuando se desea una versión más precisa y eficiente que Momentum.
- ▶ Útil en redes profundas con mínimos complicados.

Ilustración de la actualización con Nesterov Accelerated Gradient (NAG):

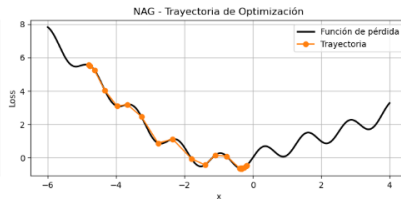
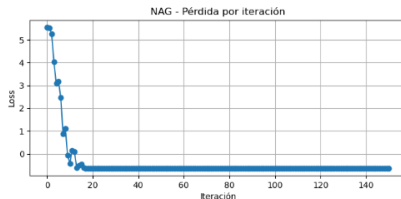
$$\begin{aligned}v_t &\leftarrow \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta_t - \gamma v_{t-1}) \\ \theta_{t+1} &\leftarrow \theta_t - v_t\end{aligned}$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



AdaGrad.

- ▶ Algoritmo adaptativo que ajusta la tasa de aprendizaje de cada parámetro individualmente.
- ▶ Acumula el cuadrado de los gradientes pasados, lo que reduce la tasa de aprendizaje para parámetros con gradientes grandes y la mantiene alta para parámetros con gradientes pequeños.

Actualización de pesos:

$$r_t \leftarrow r_{t-1} + (\nabla_{\theta} \mathcal{L}(\theta_t))^2, \quad \text{en } t = 1, r_0 = 0$$
$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{r_t + \varepsilon}} \cdot \nabla_{\theta} \mathcal{L}(\theta_t)$$

Parámetros:

- ▶ η : tasa de aprendizaje inicial.
- ▶ ε : pequeño valor para evitar división por cero ($\sim 10^{-8}$).

Hiperparámetros: Optimizadores.

Ventajas:

- ▶ Ajusta automáticamente la tasa de aprendizaje para cada parámetro.
- ▶ Es especialmente útil para datos dispersos (como en procesamiento de lenguaje natural).
- ▶ No requiere reajustar manualmente la tasa de aprendizaje durante el entrenamiento.

Desventajas:

- ▶ La tasa de aprendizaje puede volverse extremadamente pequeña a medida que se acumulan los gradientes.
- ▶ Esto puede llevar a una convergencia muy lenta o incluso al estancamiento.

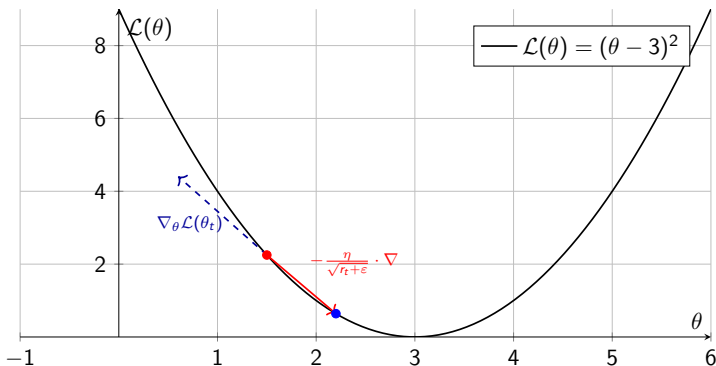
Uso recomendado:

- ▶ Cuando se trabaja con datos dispersos.
- ▶ No recomendado para entrenamiento prolongado sin reinicio o corrección (como RMSprop o Adam).

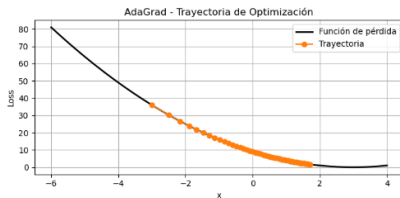
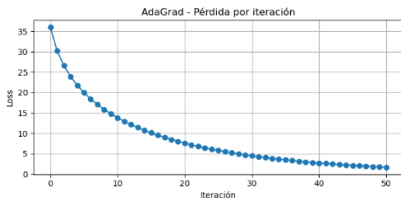
Ilustración de la actualización con AdaGrad:

$$r_t \leftarrow r_{t-1} + (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

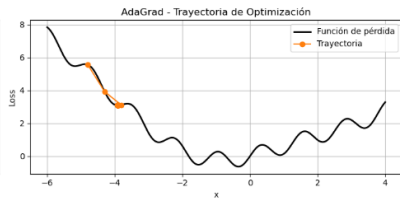
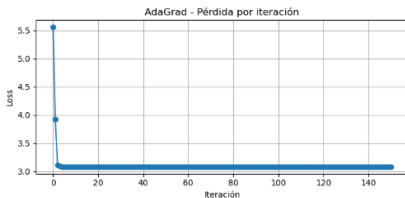
$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{r_t + \varepsilon}} \cdot \nabla_{\theta} \mathcal{L}(\theta_t)$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



RMSprop.

- ▶ Algoritmo adaptativo que corrige una limitación de AdaGrad: la disminución excesiva de la tasa de aprendizaje.
- ▶ Utiliza una media móvil del cuadrado de los gradientes para ajustar dinámicamente la tasa de aprendizaje de cada parámetro.

Actualización de pesos:

$$r_t \leftarrow \rho r_{t-1} + (1 - \rho) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$
$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{r_t + \varepsilon}} \cdot \nabla_{\theta} \mathcal{L}(\theta_t)$$

Parámetros:

- ▶ η : tasa de aprendizaje inicial.
- ▶ ρ : factor de decaimiento (típicamente $\sim 0,9$).
- ▶ ε : constante pequeña para estabilidad numérica ($\sim 10^{-8}$).

Hiperparámetros: Optimizadores.

Ventajas:

- ▶ Controla la magnitud de la tasa de aprendizaje al evitar acumulaciones excesivas.
- ▶ Permite entrenar redes profundas de manera estable.
- ▶ Funciona bien en problemas no estacionarios y datos secuenciales.

Desventajas:

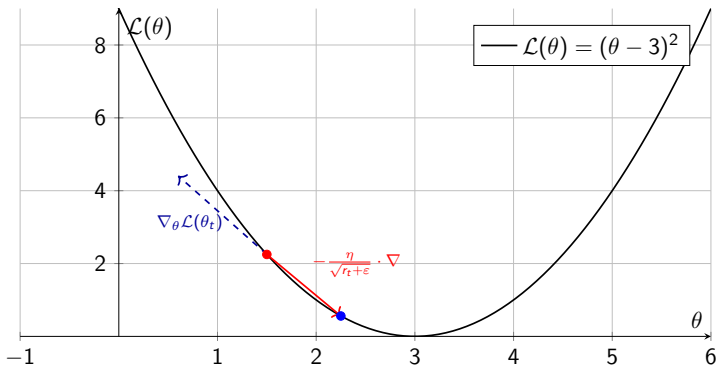
- ▶ Introduce nuevos hiperparámetros (ρ y ϵ) que requieren ajuste.
- ▶ El comportamiento puede ser sensible a la inicialización de los parámetros.

Uso recomendado:

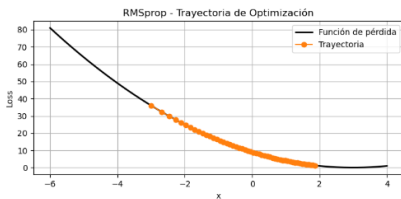
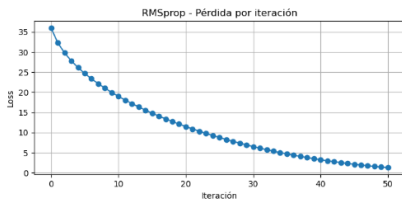
- ▶ Recomendado para redes recurrentes y problemas con gradientes altamente variables.
- ▶ Alternativa efectiva a AdaGrad y base del algoritmo Adam.

Ilustración de la actualización con RMSprop:

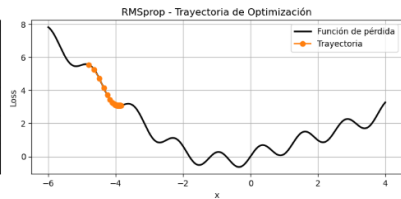
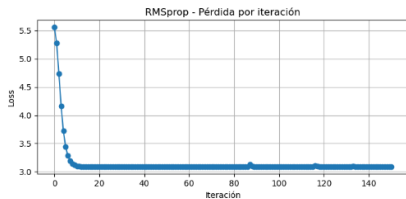
$$r_t \leftarrow \rho r_{t-1} + (1 - \rho) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$
$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{r_t} + \varepsilon} \cdot \nabla_{\theta} \mathcal{L}(\theta_t)$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



Adam (Adaptive Moment Estimation).

- ▶ Combina las ideas de Momentum y RMSprop.
- ▶ Calcula una media móvil del primer momento (velocidad) y del segundo momento (cuadrado del gradiente), con correcciones por sesgo.

Actualización de pesos:

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t), \quad (m_0 = 0)$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

$$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \cdot \hat{m}_t$$

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ β_1 : decaimiento del momento ($\sim 0,9$).
- ▶ β_2 : decaimiento del momento cuadrático ($\sim 0,999$).
- ▶ ϵ : pequeño valor para estabilidad numérica ($\sim 10^{-8}$).

Ventajas:

- ▶ Combina lo mejor de Momentum y RMSprop.
- ▶ Adaptativo, eficiente y robusto frente a diferentes escalas de los parámetros.
- ▶ Requiere poca sintonización de hiperparámetros.

Desventajas:

- ▶ Puede converger a soluciones subóptimas en algunos casos.
- ▶ Sensible a la elección de la tasa de aprendizaje inicial.

Uso recomendado:

- ▶ **Uno de los optimizadores más utilizados en redes neuronales profundas.**
- ▶ Excelente elección por defecto para una amplia gama de tareas.

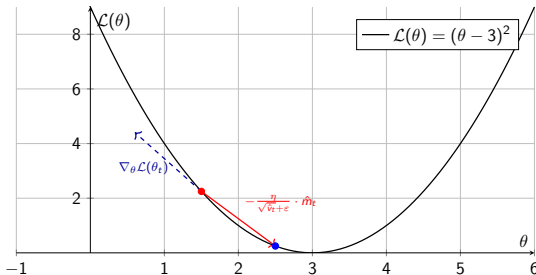
Ilustración de la actualización con Adam:

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t)$$

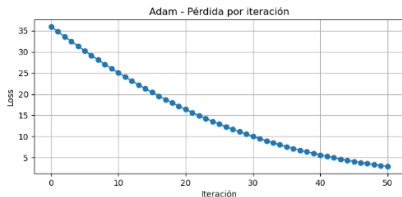
$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

$$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$$

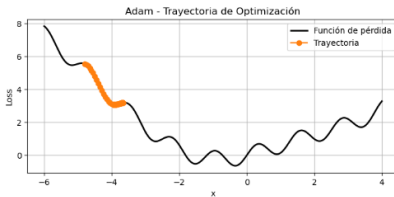
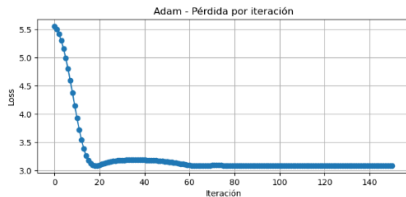
$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \cdot \hat{m}_t$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



AdamW (Adam con Weight Decay).

- ▶ Variante de Adam que desacopla explícitamente la regularización $\mathcal{L}_2$ (decaimiento de pesos) del cálculo del gradiente.
- ▶ Mejora la generalización y evita interferencias entre optimización y regularización.

Actualización de pesos:

$$\begin{aligned}m_t &\leftarrow \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t) \\v_t &\leftarrow \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2 \\ \hat{m}_t &\leftarrow \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &\leftarrow \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda \theta_t \right)\end{aligned}$$

Parámetros:

- ▶ η : tasa de aprendizaje.
- ▶ β_1, β_2 : factores de decaimiento para los momentos ($\sim 0,9, 0,999$).
- ▶ ϵ : valor pequeño para estabilidad numérica.
- ▶ λ : coeficiente de decaimiento de pesos (regularización $\mathcal{L}_2$).

Hiperparámetros: Optimizadores.

Ventajas:

- ▶ Separa correctamente la regularización $\mathcal{L}_2$ del término de gradiente.
- ▶ Mejora la capacidad de generalización en comparación con Adam.
- ▶ Mantiene la eficiencia y robustez de Adam.

Desventajas:

- ▶ Introduce el parámetro adicional λ que requiere ajuste cuidadoso.
- ▶ Menos conocido que Adam estándar (aunque ampliamente adoptado en práctica moderna).

Uso recomendado:

- ▶ Preferido sobre Adam cuando se usa regularización $\mathcal{L}_2$ explícita.
- ▶ Utilizado por defecto en modelos recientes como BERT, Vision Transformers, etc.

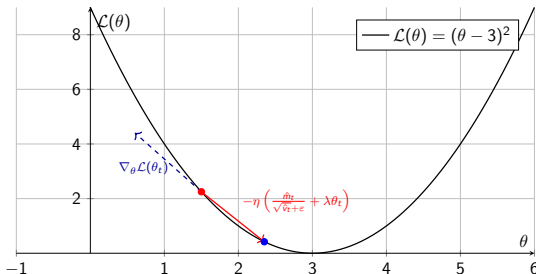
Ilustración de la actualización con AdamW:

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t)$$

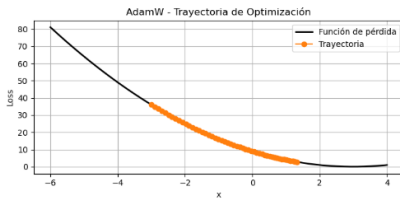
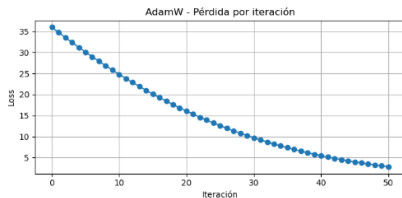
$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

$$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$$

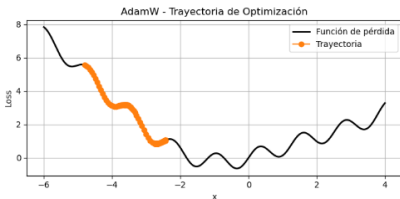
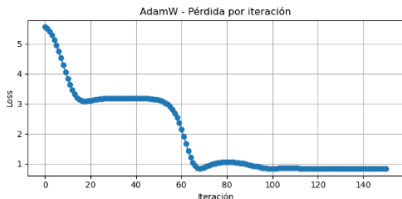
$$\theta_{t+1} \leftarrow \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda \theta_t \right)$$



Ejemplo con una función de pérdida convexa.



Ejemplo con una función de pérdida no convexa.



Comparación de Optimizadores.

Optimizador	Velocidad	Estabilidad	Recomendado para
SGD	Media	Baja	Problemas simples
Momentum	Alta	Media	Datasets complejos
NAG	Alta	Alta	Trayectorias no lineales
AdaGrad	Baja	Media	Datos dispersos
RMSprop	Alta	Alta	RNNs, señales no estacionarias
Adam	Muy alta	Muy alta	General deep learning
AdamW	Muy alta	Alta	Regularización profunda

Ver código python.

Criterios de parada en técnicas de optimización.

- ▶ Determinan cuándo detener el proceso de entrenamiento para evitar cómputo innecesario o sobreajuste.

Criterios comunes:

- ▶ Número máximo de épocas.
- ▶ Estancamiento de la función de pérdida.
- ▶ Detención temprana (early stopping).
- ▶ Norma del gradiente pequeña.
- ▶ Tiempo máximo de ejecución.

Número máximo de épocas.

- ▶ Se detiene el entrenamiento después de un número fijo de pasadas por el conjunto de entrenamiento.
- ▶ Es el criterio más simple y más utilizado como base para otros.

Condición:

$$\text{épocas realizadas} \geq \text{épocas}_{\text{máx}}$$

Parámetros:

- ▶ $\text{épocas}_{\text{máx}}$: valor fijo, típicamente entre 50 y 200, dependiendo del problema.
- ▶ Un número muy alto puede causar sobreajuste.

Estancamiento de la función de pérdida.

- ▶ El entrenamiento se detiene cuando la diferencia entre la pérdida en pasos sucesivos es menor que un umbral.
- ▶ Indica que se ha alcanzado una zona plana o un mínimo local.

Condición:

$$|\mathcal{L}_t - \mathcal{L}_{t-1}| < \varepsilon$$

Parámetros:

- ▶ $\mathcal{L}_t$: pérdida en la iteración actual.
- ▶ ε : umbral de tolerancia, típicamente 10^{-4} o menor.

Detención temprana (early stopping).

- ▶ Se monitorea la pérdida en un conjunto de validación durante el entrenamiento.
- ▶ Si no mejora tras varias iteraciones, se detiene para evitar sobreajuste.

Condición:

- ▶ Si la pérdida de validación $\mathcal{L}_{val}$ no mejora en N épocas consecutivas, detener.

Ventajas:

- ▶ Reduce el riesgo de sobreajuste.
- ▶ Ajuste dinámico de la duración del entrenamiento.

Norma del gradiente pequeña.

- ▶ Se detiene el entrenamiento si la norma del gradiente es muy pequeña.
- ▶ Implica que los parámetros han alcanzado una región de poco cambio.

Condición:

$$\|\nabla_{\theta}\mathcal{L}(\theta)\| < \delta$$

Parámetros:

- ▶ δ : umbral mínimo (por ejemplo, 10^{-5}).
- ▶ $\|\cdot\|$: norma euclídeana del vector gradiente.

Tiempo máximo de ejecución.

- ▶ Se interrumpe el entrenamiento al superar un límite de tiempo predefinido.
- ▶ Es útil en sistemas en producción, ambientes en la nube o entrenamiento distribuido.

Condición:

$$\text{tiempo transcurrido} \geq \text{tiempo}_{\text{máx}}$$

Parámetros:

- ▶ $\text{tiempo}_{\text{máx}}$: duración máxima permitida, típicamente en segundos o minutos.
- ▶ Controlado por el entorno de ejecución (por ejemplo, temporizadores del sistema).

Selección de la tasa de aprendizaje η .

- ▶ La elección de hiperparámetros afecta directamente la velocidad de convergencia y la estabilidad del entrenamiento.

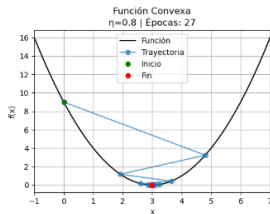
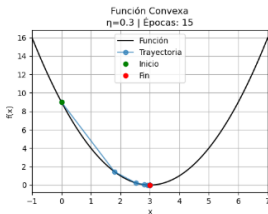
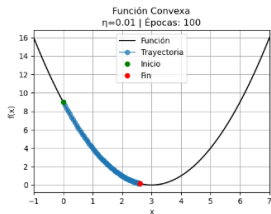
Recomendaciones generales:

- ▶ Valores típicos entre 10^{-4} y 10^{-1} .
 - ▶ Muy alta $\Rightarrow$ divergencia;
 - ▶ muy baja $\Rightarrow$ convergencia lenta.

Ejemplo del comportamiento de descenso por gradiente sobre una función convexa.

$$f(x) = (x - 3)^2$$

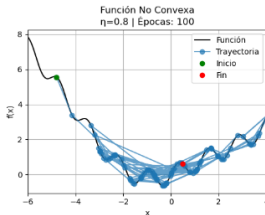
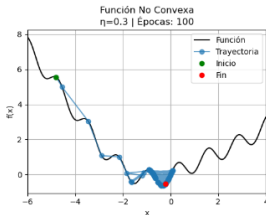
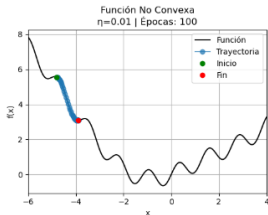
- ▶ La función tiene un único mínimo global.
- ▶ Se define tres tasas de aprendizaje $\eta = [0.01, 0.3, 0.8]$.



Ejemplo del comportamiento de descenso por gradiente sobre una función no convexa.

$$f(x) = \sin(3x) \cos(2x) + 0,2x^2$$

- ▶ La función presenta múltiples mínimos locales.
- ▶ Se define tres tasas de aprendizaje $\eta = [0.01, 0.3, 0.8]$.



Técnicas para seleccionar la tasa de aprendizaje η

- ▶ El escaneo logarítmico es una técnica simple y efectiva para seleccionar una tasa de aprendizaje adecuada η antes del entrenamiento completo de un modelo.
- ▶ Se prueban distintos valores de η en escala logarítmica, típicamente en el conjunto:

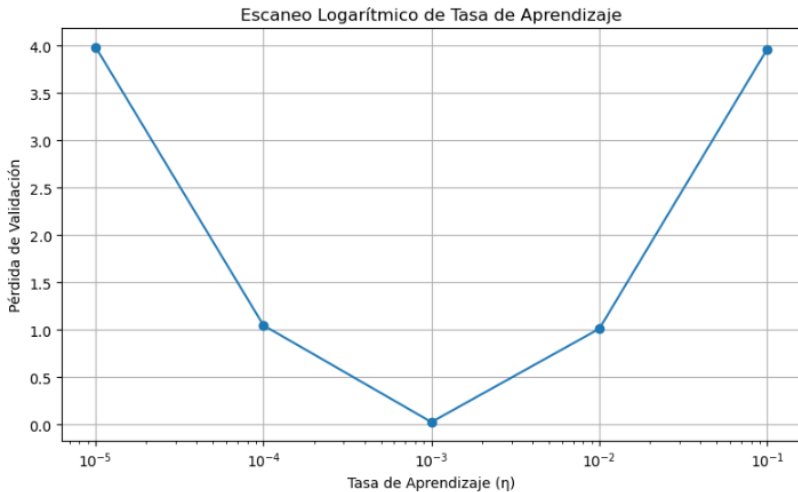
$$\eta \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$$

- ▶ Para cada valor, se entrena un modelo (parcialmente o completamente) y se registra la pérdida en validación.
- ▶ Se selecciona el valor de η que produce la menor pérdida.

Ventajas:

- ▶ Útil cuando se tiene poco conocimiento previo del modelo o del problema.
- ▶ Reduce significativamente el número de experimentos necesarios en la selección de hiperparámetros.

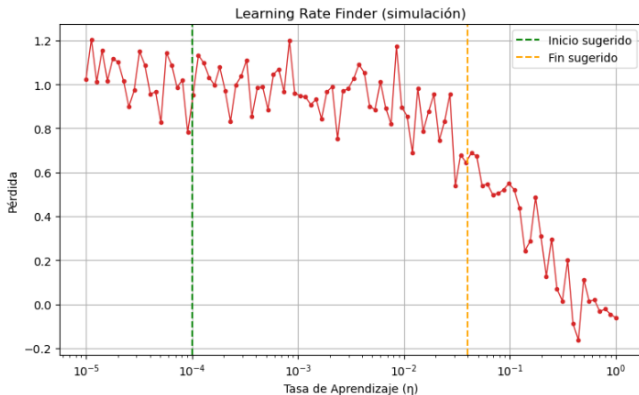
Ilustración.



Hiperparámetros: Tasa de aprendizaje η .

Técnicas para seleccionar la tasa de aprendizaje η

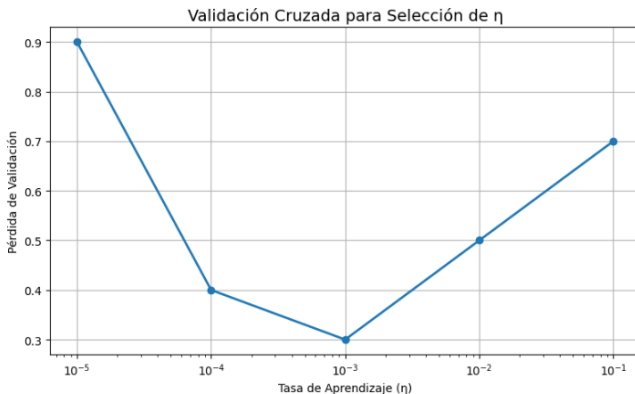
- **Learning Rate Finder:** iniciar con η pequeño y aumentarlo exponencialmente mientras se registra la pérdida; elegir el rango donde la pérdida disminuye rápidamente sin explotar.



Hiperparámetros: Tasa de aprendizaje η .

Técnicas para seleccionar la tasa de aprendizaje η

- **Validación cruzada o hold-out:** entrenar modelos con diferentes η y seleccionar el valor que mejor generalice (mínima pérdida en validación).



Técnicas para seleccionar la tasa de aprendizaje η

- ▶ **Reducción programada (Scheduled decay):** disminuir η a lo largo del tiempo para afinar el entrenamiento.

Reducción escalonada.

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}, \quad \text{donde } \gamma < 1$$

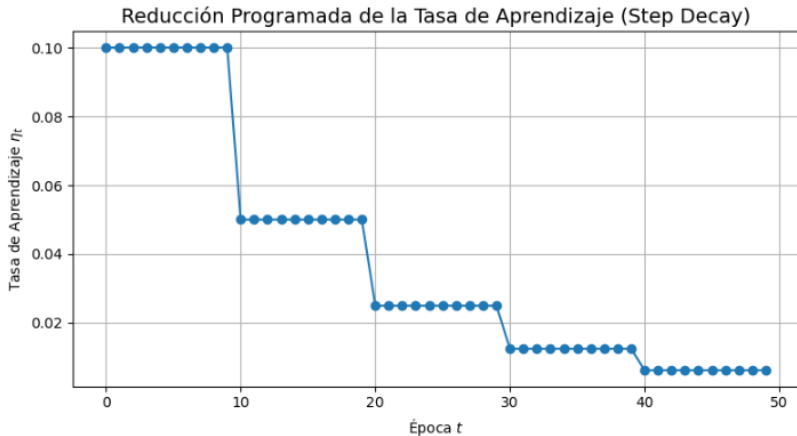
Características:

- ▶ Disminuye η cada s iteraciones (o épocas) de forma controlada.
- ▶ Típicamente usada para afinar el entrenamiento conforme se avanza.
- ▶ Muy común en **SGD**, **Momentum** y **NAG**.

Ventajas:

- ▶ Fácil de implementar.
- ▶ Reduce el riesgo de sobrepasar el mínimo.

Ilustración.



Técnicas para seleccionar la tasa de aprendizaje η

- ▶ **Reducción por desempeño:** reducir η si la pérdida de validación se estanca o aumenta (*ReduceLROnPlateau*).

Reducción condicionalo.

Si $\mathcal{L}_{\text{val}}(t)$ no mejora en k épocas $\Rightarrow \eta \leftarrow \eta \cdot \gamma$

Características:

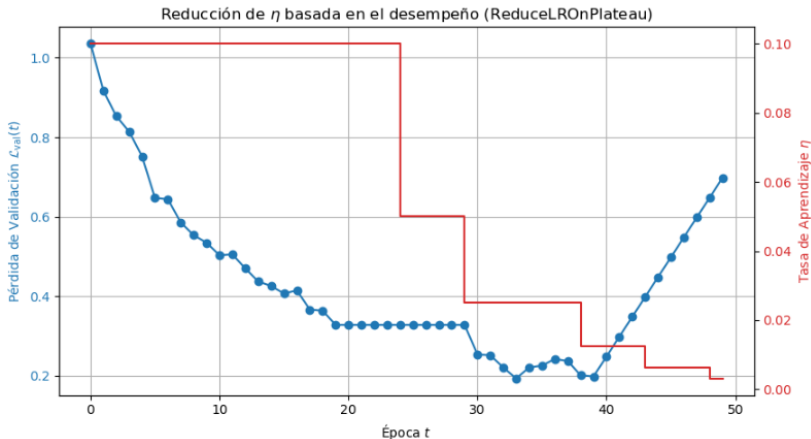
- ▶ Detecta mesetas o estancamiento durante el entrenamiento.
- ▶ Automáticamente ajusta η cuando no hay mejora en el conjunto de validación.
- ▶ Muy usada en **RMSprop**, **Adam** y **AdamW**.

Ventajas:

- ▶ Reduce el número de hiperparámetros fijos.
- ▶ Se adapta dinámicamente al comportamiento de la red.

Ilustración:

Si $\mathcal{L}_{\text{val}}(t)$ no mejora en $k(=5)$ épocas $\Rightarrow \eta \leftarrow \eta \cdot \gamma$



Uso de optimizadores adaptativos.

- ▶ Algoritmos como AdaGrad, RMSprop y Adam ajustan automáticamente η por parámetro.
- ▶ Menos sensibles a la elección inicial de la tasa de aprendizaje.

Ejemplo (Adam):

$$\theta_t \leftarrow \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

- ▶ $\hat{m}_t$: estimación del primer momento (media).
- ▶ $\hat{v}_t$: estimación del segundo momento (varianza).

Buenas prácticas:

- ▶ Comenzar con $\eta = 10^{-3}$ si se usa Adam; $\eta = 10^{-2}$ o 10^{-1} con SGD.
- ▶ Utilizar **early stopping** para evitar sobreentrenamiento.
- ▶ Reducir η si la pérdida oscila mucho o no disminuye.
- ▶ Aplicar **reducción programada** o **scheduler** si el entrenamiento se estanca.

Recomendación:

- ▶ Usar un **learning rate finder** en conjunto con validación cruzada para elegir una tasa adecuada.

Selección del parámetro γ en Momentum.

- ▶ Es el coeficiente de inercia: controla cuánto del paso anterior se conserva.
- ▶ Rango típico: $0 < \gamma < 1$.

$$v_t \leftarrow \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$
$$\theta_{t+1} \leftarrow \theta_t - v_t$$

Valores recomendados:

- ▶ **Problemas estándar:** $\gamma = 0,9$ (valor por defecto en muchas librerías).
- ▶ **Datos ruidosos:** $\gamma = 0,8$ o menos (menos acumulación).
- ▶ **Redes profundas:** $\gamma = 0,95$ a $0,99$ (mejor aceleración).

Recomendaciones prácticas:

- ▶ Si la pérdida oscila mucho $\Rightarrow$ reducir γ .
- ▶ Si el aprendizaje es muy lento $\Rightarrow$ aumentar γ .
- ▶ Siempre ajustar junto con η .

Selección del parámetro ρ en RMSprop.

- ▶ Es el coeficiente de decaimiento exponencial.
- ▶ Controla la memoria del promedio de gradientes pasados.
- ▶ Valores altos suavizan más (mayor memoria), valores bajos reaccionan más rápido.

$$v_t = \rho v_{t-1} + (1 - \rho) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \varepsilon} \nabla_{\theta} \mathcal{L}(\theta_t)$$

Valores típicos recomendados:

- ▶ General: $\rho = 0,9$ (valor por defecto).
- ▶ Datos ruidosos o gradientes inestables: $\rho \in [0,95, 0,99]$.
- ▶ Problemas que requieren respuesta rápida a cambios: $\rho \in [0,8, 0,9]$.

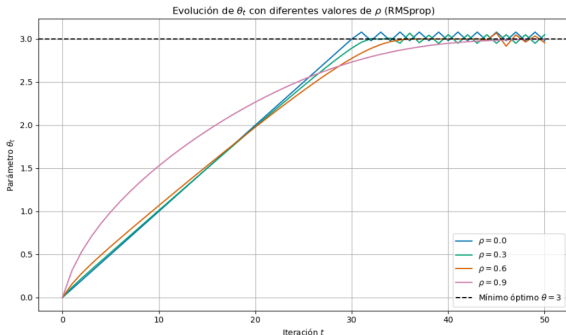
Recomendaciones prácticas:

- ▶ No usar $\rho < 0,7$: puede hacer el promedio inestable.
- ▶ Ajustar en conjunto con η y el tamaño del mini-lote.
- ▶ Verificar la estabilidad observando la pérdida en validación.

Hiperparámetros: Parámetro ρ en RMSprop.

Análisis del efecto de ρ : (función $\mathcal{L}(\theta) = (\theta - 3)^2$)

- ▶ $\rho = 0$: no hay promedio; se comporta como SGD con normalización puntual.
- ▶ ρ **bajo (e.g., 0.3)**: adaptación rápida al gradiente actual, pero con más inestabilidad..
- ▶ ρ **alto (e.g., 0.9)**: adaptación lenta pero más suave, con memoria larga.



Ver código python.

Selección de los parámetros β_1 y β_2 en Adam.

- ▶ β_1 : controla la memoria del gradiente (similar a Momentum).
- ▶ β_2 : controla la memoria del cuadrado del gradiente (escalado adaptativo).

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Valores típicos:

- ▶ $\beta_1 = 0,9$: valor por defecto, funciona bien en la mayoría de los casos.
- ▶ $\beta_2 = 0,999$: recomendado para evitar pasos inestables.

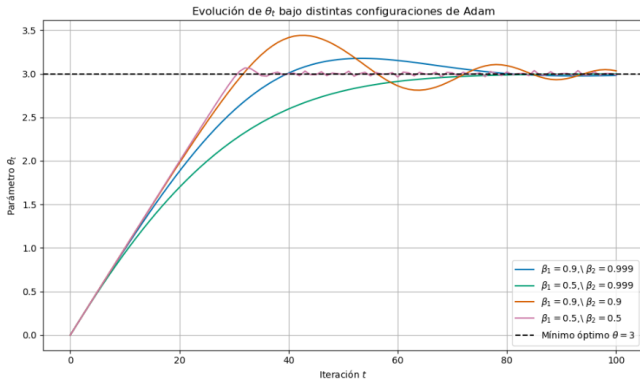
Recomendaciones prácticas:

- ▶ Reducir β_1 si el gradiente cambia rápidamente.
- ▶ Reducir ligeramente β_2 si se necesita adaptación más rápida.
- ▶ Ajustar en conjunto con η y el tamaño del mini-lote.

Hiperparámetros: Parámetros β_1 y β_2 en Adam.

Análisis del efecto de β_1 y β_2 : (función $\mathcal{L}(\theta) = (\theta - 3)^2$)

- ▶ Valores altos de β_1 y β_2 suavizan más la actualización.
- ▶ Valores bajos responden más rápidamente pero pueden producir oscilaciones.



Ver código python.

Hiperparámetro de regularización λ .

- ▶ La regularización es una técnica que modifica la función de pérdida para controlar la complejidad del modelo.
- ▶ Se añade un término que penaliza los modelos con pesos grandes o estructuras demasiado complejas.
- ▶ El objetivo es mejorar la capacidad de generalización y prevenir el sobreajuste (*overfitting*).

Hiperparámetro de regularización λ .

Función de Pérdida Regularizada.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{datos}} + \lambda \cdot \mathcal{L}_{\text{reg}}$$

- ▶ $\mathcal{L}_{\text{datos}}$: pérdida asociada al ajuste a los datos.
- ▶ $\mathcal{L}_{\text{reg}}$: penalización sobre los pesos.
- ▶ El hiperparámetro λ es un número real no negativo: $\lambda \geq 0$.
Se exploran comúnmente valores en el rango:
$$\lambda \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$$
 - ▶ $\lambda = 0$: no hay regularización. Riesgo elevado de **sobreajuste**.
 - ▶ Valores muy pequeños reducen ligeramente los pesos, manteniendo la flexibilidad del modelo.
 - ▶ Valores intermedios logran equilibrio entre ajuste y simplicidad.
 - ▶ Valores grandes simplifican excesivamente el modelo, pudiendo causar **subajuste**.

Validación Cruzada.

- ▶ Consiste en dividir el conjunto de datos en k folds.
- ▶ Para cada valor candidato de λ , por ejemplo:

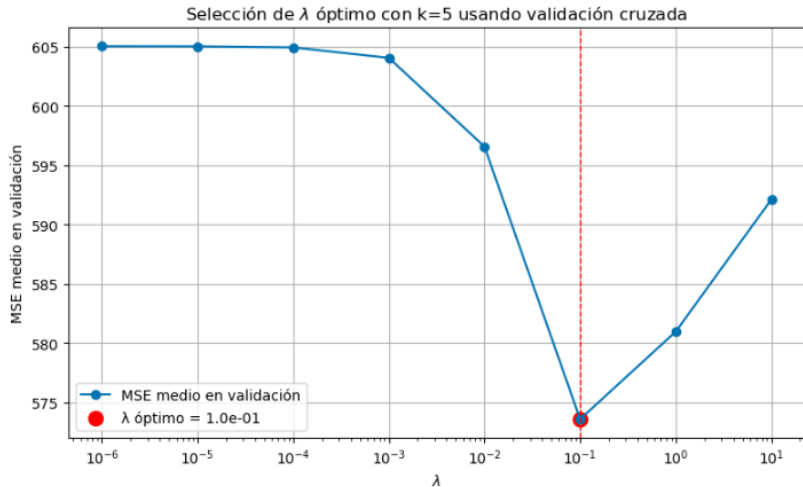
$$\lambda \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\},$$

se entrena el modelo k veces.

- ▶ En cada entrenamiento se calcula el error medio de validación.
- ▶ Se selecciona el valor con mejor rendimiento:

$$\lambda_{\text{óptimo}} = \arg \min_{\lambda} \frac{1}{k} \sum_{i=1}^k \mathcal{L}_{\text{val}}^{(i)}$$

Selección de λ usando Validación Cruzada.



Curvas de Validación.

- ▶ Se entrena el modelo para distintos valores de λ (usualmente en escala logarítmica).
- ▶ Para cada λ , se calcula el error medio de entrenamiento y validación frente a λ , usando validación cruzada:

$$\text{MSE}_{\text{val}}(\lambda) = \frac{1}{k} \sum_{i=1}^k \mathcal{L}_{\text{val}}^{(i)}(\lambda)$$

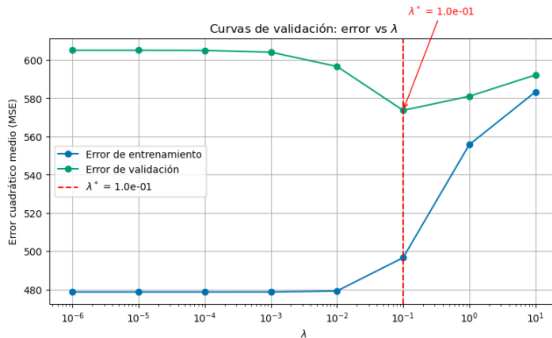
- ▶ Se selecciona el valor que minimiza el error de validación:

$$\lambda^* = \arg \min_{\lambda} \text{MSE}_{\text{val}}(\lambda)$$

- ▶ El valor λ^* logra el mejor compromiso entre sesgo y varianza.

Hiperparámetro de regularización λ .

Selección de λ usando Curvas de Validación.



	Error de entrenamiento	Error de validación
¿Dónde se calcula?	Sobre los datos usados para ajustar el modelo	Sobre datos nunca vistos durante el ajuste
¿Qué mide?	Capacidad del modelo para “memorizar”	Capacidad del modelo para generalizar
¿Esperado?	Suele ser bajo, especialmente si el modelo es flexible	Más alto que el de entrenamiento, refleja realismo

Búsqueda Exhaustiva (Grid Search).

- ▶ Se define un conjunto discreto de valores posibles para λ .

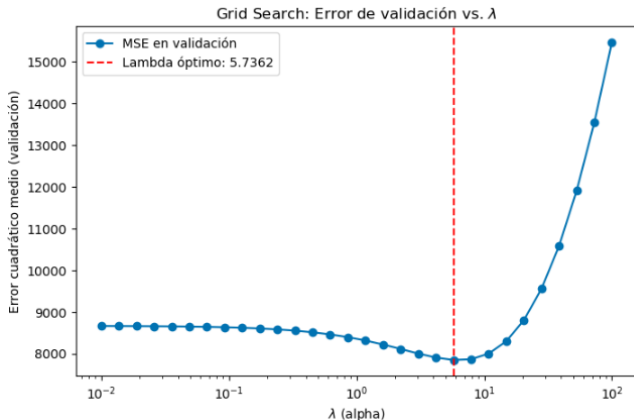
$$\{\lambda_1, \lambda_2, \dots, \lambda_n\}$$

- ▶ Se entrena el modelo para cada valor, usando validación cruzada.
- ▶ Se selecciona el valor de λ que logra el menor error.

Hiperparámetro de regularización λ .

Selección de λ usando Búsqueda Exhaustiva (Grid Search).

30 valores de λ distribuidos logarítmicamente entre: 0.01 y 100.



Ver código python.

Hiperparámetro número de épocas.

- ▶ El número de épocas (**epochs**) define cuántas veces el algoritmo de entrenamiento recorre por completo el conjunto de datos de entrenamiento.
- ▶ Es un hiperparámetro fundamental que influye directamente en el rendimiento del modelo.
- ▶ Cada época permite que el modelo actualice sus pesos basándose en todos los ejemplos de entrenamiento.

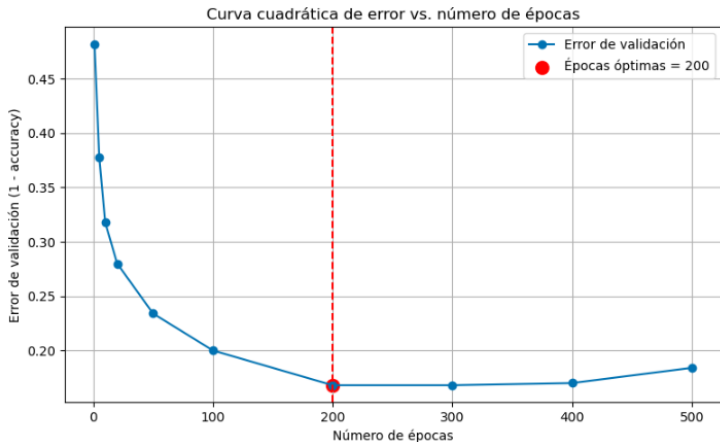
Importancia del número de épocas.

- ▶ Un número insuficiente de épocas puede causar **subajuste** (underfitting): el modelo no aprende correctamente los patrones.
- ▶ Un número excesivo de épocas puede provocar **sobreajuste** (overfitting): el modelo aprende demasiado bien el conjunto de entrenamiento, perdiendo capacidad de generalización.
- ▶ Elegir adecuadamente el número de épocas permite equilibrar estos dos fenómenos.

Técnicas para seleccionar el número de épocas.

- ▶ **Validación cruzada:** entrenar varios modelos con diferentes números de épocas y medir el error de validación.
- ▶ **Curvas de aprendizaje:** observar cómo evoluciona el error de validación frente al número de épocas.
- ▶ **Early stopping:** detener el entrenamiento cuando el error de validación deja de mejorar durante un número determinado de épocas consecutivas.

Ilustración de la técnica: Validación cruzada.

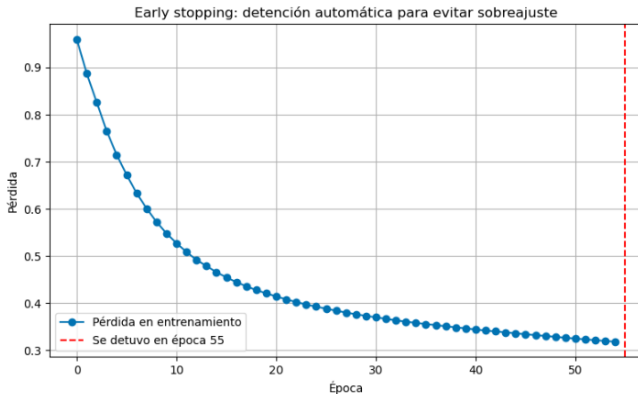


Early stopping.

- ▶ Consiste en monitorizar el error de validación durante el entrenamiento.
- ▶ Se detiene el entrenamiento si el error de validación no mejora después de un número determinado de épocas consecutivas.
- ▶ Evita el sobreajuste y reduce el tiempo de entrenamiento.

Ilustración de la técnica: Early stopping.

- ▶ Se detiene automáticamente si el error de validación no mejora durante 10 épocas consecutivas.



Ver código python.

Hiperparámetro tamaño de mini-lote.

- ▶ El tamaño de mini-lote (**batch size**) es el número de muestras que se utilizan para calcular y aplicar una actualización de los pesos durante el entrenamiento.
- ▶ Se sitúa entre dos extremos:
 - ▶ **Stochastic Gradient Descent (SGD):**
batch size = 1 (una muestra).
 - ▶ **Batch Gradient Descent:**
batch size = número total de muestras.

Rol del tamaño de mini-lote.

- ▶ Controla el equilibrio entre la frecuencia de actualización de los pesos y la precisión en la estimación del gradiente.
- ▶ Afecta el tiempo de entrenamiento, el uso de memoria y la calidad del modelo final.
- ▶ Es un hiperparámetro importante que suele ajustarse experimentalmente.

Efectos de batch size pequeño.

- ▶ Introduce mayor **aleatoriedad** en las actualizaciones.
- ▶ Puede ayudar a escapar de mínimos locales y zonas planas.
- ▶ Menor uso de memoria, adecuado para dispositivos con recursos limitados.
- ▶ Actualizaciones más frecuentes, pero menos precisas.

Efectos de batch size grande.

- ▶ Estimaciones del gradiente más estables y precisas.
- ▶ Mejor aprovechamiento del paralelismo en GPU.
- ▶ Mayor necesidad de memoria.
- ▶ Riesgo de caer en mínimos locales o zonas poco profundas de la función de pérdida.

Técnicas para elegir el tamaño de mini-lote.

- ▶ **Validación cruzada:**

Entrenar modelos con diferentes valores y comparar resultados.

- ▶ **Experimentación empírica:**

Probar tamaños como 16, 32, 64, 128, 256, etc.

- ▶ **Ajustar según la memoria disponible:**

Elegir el mayor valor que permita la GPU o CPU sin desbordamiento.

- ▶ **Considerar la interacción con otros hiperparámetros como la tasa de aprendizaje.**

Optimización de Hiperparámetros.

- ▶ En redes neuronales profundas, el rendimiento depende fuertemente de:
 - ▶ Tasa de aprendizaje η
 - ▶ Número de capas y neuronas
 - ▶ Regularización λ
 - ▶ Tamaño de batch
- ▶ Surge el problema de optimización de los hiperparámetros:

$$\theta^* = \arg \min_{\theta \in \Theta} \mathcal{L}(\theta)$$

donde θ representa el conjunto de hiperparámetros.

Grid Search

Explora exhaustivamente todas las combinaciones posibles en una grilla discreta.

Si cada hiperparámetro tiene k valores y existen d hiperparámetros:

$$\text{Total de evaluaciones} = k^d$$

Uso actual

- ▶ Conceptualmente simple
- ▶ Costoso en alta dimensión
- ▶ Poco eficiente en Deep Learning

Random Search

Muestrea configuraciones aleatorias del espacio de búsqueda.

En lugar de recorrer una grilla:

$$\theta_i \sim \mathcal{U}(\Theta)$$

Ventajas

- ▶ Explora mejor espacios de alta dimensión
- ▶ Más eficiente cuando pocos hiperparámetros son relevantes
- ▶ Muy usado en la práctica

Bayesian Optimization

Construye un modelo probabilístico del rendimiento:

$$f(\theta) \approx \mathcal{L}(\theta)$$

Utiliza:

- ▶ Proceso Gaussiano o modelos sustitutos
- ▶ Función de adquisición (Expected Improvement)

Uso actual

- ▶ Muy usado en investigación
- ▶ Reduce número de evaluaciones costosas

Hyperband / ASHA

Asignación adaptativa de recursos durante el entrenamiento.

Idea central:

- ▶ Evaluar muchas configuraciones con pocos recursos
- ▶ Eliminar tempranamente las peores
- ▶ Asignar más recursos a las prometedoras

Uso actual

- ▶ Muy popular en Deep Learning moderno
- ▶ Eficiente cuando el entrenamiento es costoso

Comparación General

Método	Exploración	Costo	Uso actual
Grid Search	Exhaustiva	Alto	Bajo en DL
Random Search	Aleatoria	Medio	Muy usado
Bayesian Opt.	Guiada	Bajo	Investigación
Hyperband/ASHA	Adaptativa	Bajo	Muy popular

Optuna: Optimización Automática de Hiperparámetros

- ▶ **Optuna** es una librería moderna para la optimización automática de hiperparámetros.
- ▶ Permite encontrar configuraciones óptimas sin exploración exhaustiva.
- ▶ Se basa principalmente en métodos de **optimización bayesiana**.

Objetivo matemático:

$$\theta^* = \arg \min_{\theta \in \mathcal{H}} f(\theta)$$

- ▶ donde:
 - ▶ θ representa los hiperparámetros.
 - ▶ $\mathcal{H}$ es el espacio de búsqueda.
 - ▶ $f(\theta)$ es la función de validación.

Componentes Fundamentales

1. Study

- ▶ Representa el proceso completo de optimización.
- ▶ Define si se desea minimizar o maximizar la función objetivo.

2. Trial

- ▶ Cada evaluación individual de un conjunto de hiperparámetros.
- ▶ En cada trial se generan valores mediante:
 - ▶ `suggest_float`
 - ▶ `suggest_int`
 - ▶ `suggest_categorical`

3. Sampler

- ▶ Algoritmo que decide cómo explorar el espacio de búsqueda.
- ▶ **Por defecto utiliza TPE** (Tree-structured Parzen Estimator).

Optimización Bayesiana con TPE

- ▶ El algoritmo TPE modela: $p(\theta|y)$
donde:
 - ▶ θ son los hiperparámetros.
 - ▶ y es el valor de la función objetivo.
- ▶ Se construyen dos distribuciones:

$$l(\theta) = p(\theta|y < y^*), \quad g(\theta) = p(\theta|y \geq y^*)$$

- ▶ El nuevo conjunto de hiperparámetros se selecciona maximizando:

$$\frac{l(\theta)}{g(\theta)}$$

Ventaja: prioriza regiones prometedoras del espacio de búsqueda.

Ejemplo Básico de Uso

```
import optuna

def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
    n_layers = trial.suggest_int("n_layers", 1, 4)

    loss = entrenar_modelo(lr, n_layers)
    return loss

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=50)

print(study.best_params)
```

Ver código python.

Gracias por su atención.

